Dedication

I dedicate this project to:

To my dear parents, Idris and Dalel,
Your love, sacrifices, and constant support have been the pillars of my journey.
Your strength, wisdom, and belief in me have shaped who I am.
This achievement is as much yours as it is mine.

To my sisters, Asma, Soumaya, Mariem, Riheb, and Amira,
Thank you for your endless support, for always standing by my side,
and for bringing warmth and joy into my life. Each of you inspires me in your own way.

To my friends,
Your encouragement and companionship have lightened the difficult moments
and made the good ones even better. I deeply appreciate your presence on this journey.

To everyone at STS,
A special thanks to Asses Anis for his guidance in the conception phase,
and to Jouili Imed and Azzabi Lassad for their valuable support during development.
I am grateful to all the members of the STS company for their help, kindness, and collaboration.

To all of you, thank you from the bottom of my heart.

Acknowledgements

I would like to express my sincere gratitude to all those who contributed to the success of my internship and supported me during the writing of this report.

I extend my warmest thanks to my company supervisors, **Mr. Anis Assess** and **Mr. Imed Jouili**, for their invaluable guidance, availability, and support throughout the internship. Their insights and encouragement were crucial to the progress and completion of this work.

I am deeply grateful to my academic supervisor, **Mrs. Rahma Haouas**, for her continuous support, thoughtful advice, and kind encouragement. Her trust and guidance greatly enriched this experience.

I would also like to express my sincere appreciation to the **members of the jury** for the honor they have bestowed upon me by agreeing to evaluate my work.

Finally, I am especially thankful to **my family** and my **friends** for their unwavering support, motivation, and encouragement throughout the entire internship period.

Contents

1 General Project Context **3**

Introduction 3

1.1 Presentation of the Host Company 4

1.1.1 Overview and History 4

1.1.2 Operations and Scale 4

1.1.3 Services and Mission 5

1.2 Project Presentation 5

1.2.1 Problem Statement 5

1.2.2 Study of the Existing System 6

1.2.3 Proposed Solution 7

1.2.4 Objectives 8

1.3 Development Methodology 8

1.3.1 Evaluation of Methodologies 8

1.3.2 Chosen Methodology: Scrum 11

1.3.3 Scrum Roles 12

Conclusion 12

2 Sprint 0: General project planning **13**

Introduction 13

2.1 Requirements Study 14

2.1.1 Identification of Actors 14

2.1.2 Functional Requirements 14

2.1.3 Non-Functional Requirements 15

2.2 Project Management with SCRUM 16

2.2.1 Product Backlog 16

2.2.2 Sprint Planning 18

2.2.3 General Use Case Diagram 19
2.2.4 General Class Diagram 20
2.3 Technical Choices 21
2.4 Application Architecture 24
2.4.1 Logical Architecture 24
2.4.2 Physical Architecture 25
Conclusion 27

3 Study and implementation of sprint 1 28
Introduction 28
3.1 Sprint 1 Objective 28
3.2 Sprint 1 Backlog 29
3.3 Design 29
3.3.1 Sprint 1 Use Case Diagram 30
3.3.2 Class diagram 32
3.3.3 Sequence diagram 33
3.4 Implementation 33
Conclusion 37

4 Study and implementation of sprint 2 38
Introduction 38
4.1 Sprint 2 Objective 38
4.2 Sprint 2 Backlog 39
4.3 Design 39
4.3.1 Use Case Diagram 40
4.3.2 Class diagram 43
4.3.3 Sequence diagram 44
4.4 Implementation 45
Conclusion 50

5 Study and Implementation of Sprint 3 51
Introduction 51
5.1 Sprint 3 Objective 51
5.2 Sprint 3 Backlog 52
5.3 Design 52
5.3.1 Use Case Diagram 52
5.3.2 Class diagram 56
5.3.3 Sequence diagram 57

5.4 Implementation 58
Conclusion 62

6 Study and Implementation of Sprint 4 63
Introduction 63
6.1 Sprint 4 Objective 63
6.2 Sprint 4 Backlog 64
6.3 Design 64
6.3.1 Use Case Diagram 65
6.3.2 Class Diagram 67
6.3.3 Sequence Diagram 68
6.4 AI Integration 68
6.5 Implementation 70
Conclusion 73

7 Deployment Phase 74
Introduction 74
7.1 Significance of Deployment 74
7.2 Docker Containerization 75
7.2.1 Docker Compose & Dockerfile 76
7.3 Hosting Provider Comparison 78
7.4 Automated Deployment via Coolify 79
7.5 Deployment Architecture and Advantages 82
Conclusion 83

List of Figures

1.1 Tunisiebooking Official Logo 4
1.2 Company statistics 4
1.3 Tunisiebooking Home Page 6
1.4 Waterfall Methodology Process 9
1.5 Scrum Methodology Process 10
1.6 Kanban Methodology Process 10
1.7 2TUP Methodology Process 11

2.1 General Use Case Diagram 20
2.2 General Class Diagram 21
2.3 MySQL Logo 22
2.4 Ranking of backend frameworks in 2024-2025 22
2.5 Laravel Logo 23
2.6 Ranking of frontend frameworks in 2024-2025 23
2.7 Next.js Logo 24
2.8 Tailwind CSS Logo 24
2.9 MVC architecture. 25
2.10 Physical architecture. 26

3.1 Use Case Diagram – Authentication and User Management (Sprint 1) 30
3.2 Class diagram – Authentication and User Management (Sprint 1) 32
3.3 Authentication Sequence diagram 33
3.4 Registration Interfaces 34
3.5 Email Verification Interfaces 34
3.6 Login and Password Reset Request Interfaces 35
3.7 Password Reset Completion Interfaces 35
3.8 User Management Dashboard 36

3.9 User Editing and Deletion Interfaces 36
3.10 Add New User Interface 37

4.1 Use Case Diagram – Hotel Search and Booking (Sprint 2) 40
4.2 Class diagram – Hotel Search and Booking (Sprint 2) 43
4.3 Hotel Search and Booking Sequence diagram 44
4.4 Hotel Search and Booking Sequence diagram for admin 45
4.5 Search Interface 46
4.6 Hotel Browsing Interface (Admin View) 46
4.7 Hotel Details Interface 47
4.8 Hotel Browsing Interface (Client View) 48
4.9 Reservation Interface 48
4.10 Invoice and Reservation Status Interface 49
4.11 Confirmation Email 49

5.1 Use Case Diagram – admin Environment (Sprint 3) 53
5.2 Class Diagram – Administrators environment (Sprint 3) 56
5.3 Agency management Sequence diagram 57
5.4 Supplier management Sequence diagram 57
5.5 Hotel mapping Sequence diagram 58
5.6 Hotels interface 58
5.7 Administration Home interface 59
5.8 Reservations interface 60
5.9 Hotel Chain interface 60
5.10 Themes interface 60
5.11 Agency interface 61
5.12 Suppliers interface 61
5.13 Mapping interface 62

6.1 Use Case Diagram – Client Space and AI Chatbot (Sprint 4) 65
6.2 Class Diagram – Client Space and Chatbot (Sprint 4) 67
6.3 Chatbot Sequence Diagram 68
6.4 DeepSeek Logo 69
6.5 DeepSeek Chatbot Implementation Architecture 69
6.6 ChatBot interfaces 70
6.7 Client Space Home 71
6.8 Client Space Search History 71
6.9 Client Space Reservations 72

6.10 Client Favorite Hotels 72
6.11 Client Profile 73

7.1 Docker Logo 75
7.2 `docker_compose.yml` used for Laravel and MySQL containers 76
7.3 Dockerfile for the Laravel backend container 77
7.4 `docker_compose.yml` used for the Next.js frontend 77
7.5 Dockerfile for the Next.js frontend container 78
7.6 Coolify Logo 80
7.7 Coolify dashboard overview showing all running containers 80
7.8 Backend container configuration 81
7.9 Frontend container configuration 82
7.10 Database container configuration 82
7.11 Preview of the Deployed Website on testal.tunisiebooking.com 82

List of Tables

1.1 Tunisiebooking Company Profile 5
1.2 Comparison between current and proposed systems 8
1.3 Scrum Team Assignments 12

2.1 User Roles Overview 14
2.2 Product Backlog 18
2.3 Sprint Planning 19

3.1 Sprint 1 Backlog 29
3.2 textual description of the use case Authentication 31
3.3 Textual description of the use case “Reset Password”. 31
3.4 Textual description of the use case “Manage Users”. 32

4.1 Sprint 2 Backlog 39
4.2 Textual description of the use case “Consult Offers”. 41
4.3 Textual description of the use case “Browse Hotels”. 41
4.4 Textual description of the use case “Search for Available Offers”. 42
4.5 Textual description of the use case “Manage Reservations”. 42

5.1 Sprint 3 Backlog 52
5.2 Textual description of the use case “Manage Agencies”. 54
5.3 Textual description of the use case “Manage Hotels”. 54
5.4 Textual description of the use case “Manage Hotel Mappings”. 55
5.5 Textual description of the use case “Manage Suppliers”. 55

6.1 Sprint 4 Backlog 64
6.2 Textual description of the use case “Consult Client Space”. 66
6.3 Textual description of the use case “Use Chatbot”. 66

7.1 Comparison of Hosting Providers for TunisieBooking 79

List of Abbreviations

PHP Hypertext Preprocessor

AI Artificial Intelligence

MySQL My Structured Query Language

2TUP Two Track Unified Process

API Application Programming Interface

SSR Server-Side Rendering

SSG Static Site Generation

CSS Cascading Style Sheets

UI User Interface

MVC Model-View-Controller

HTML HyperText Markup Language

AWS Amazon Web Services

Admin Administrator

CSRF Cross-Site Request Forgery

GPT Generative Pre-trained Transformer

CI/CD Continuous Integration/Continuous Deployment

VPS Virtual Private Server

EC2 Elastic Compute Cloud

General Introduction

In the current digital age, the travel and tourism sector increasingly relies on innovative technological solutions to enhance user experience and streamline services. The rise of online booking platforms has transformed how people plan and manage their trips, especially in the context of international travel between neighboring countries such as Algeria and Tunisia.

This final year project is carried out in collaboration with **TunisieBooking**, a leading Tunisian travel service provider. It aims to assist a major **Algerian travel agency** in launching a specialized hotel booking platform dedicated to Algerian tourists traveling to Tunisia via the subdomain `al.tunisiebooking.com`.

The problematic addressed in this project is the lack of a centralized and user friendly system that aggregates hotel offers from multiple providers such as TunisieBooking and SamehVoyages and makes them accessible in real time for Algerian clients. The absence of such a platform complicates the booking process, often requiring users to navigate multiple websites or rely on intermediaries.

To solve this issue, the proposed solution consists of developing a **modern, responsive, and secure web application** that integrates various data sources and offers a unified interface. The platform provides real-time hotel availability through API integration, personalized recommendations, intuitive navigation, and seamless cross-device compatibility. It also allows users to manage their reservations and discover curated travel experiences tailored to their preferences.

The main objectives of this project are to design and implement a robust hotel booking system tailored to the needs of Algerian users, ensure real-time synchronization of hotel availability from multiple providers, offer a smooth and user-centric reservation process with secure authentication and efficient data handling, and facilitate intelligent travel planning through smart features like a personalized client area and an AI-powered chatbot.

The content of this report is structured into seven chapters as follows:

- **Chapter 1: General project context**
  This chapter introduces the host organization, the project context, the identified problem,

the proposed solution, and the chosen development methodology.

- **Chapter 2: Sprint 0 – General project planning**
  It presents the requirement gathering process, technical decisions, application architecture, and project planning based on the Scrum methodology.

- **Chapter 3: First Sprint**
  This chapter describes the development of the authentication module and user management functionalities.

- **Chapter 4: Second Sprint**
  It focuses on the implementation of hotel search, viewing hotel details, and handling hotel availability.

- **Chapter 5: Third Sprint**
  This chapter details the creation of a custom admin dashboard adapted to different user roles.

- **Chapter 6: Fourth Sprint**
  It introduces advanced features in the client space, including the integration of an AI-based chatbot for assistance.

- **Chapter 7: Deployment Phase**
  It covers the deployment process using Docker, a comparison of hosting solutions, and automation via Coolify.

Finally, the report concludes with a general reflection on the project outcomes and presents potential areas for improvement and future development perspectives.

Chapter 1

General Project Context

Contents

Introduction **3**

1.1 Presentation of the Host Company **4**

1.1.1 Overview and History 4

1.1.2 Operations and Scale 4

1.1.3 Services and Mission 5

1.2 Project Presentation **5**

1.2.1 Problem Statement 5

1.2.2 Study of the Existing System 6

1.2.3 Proposed Solution 7

1.2.4 Objectives 8

1.3 Development Methodology **8**

1.3.1 Evaluation of Methodologies 8

1.3.2 Chosen Methodology: Scrum 11

1.3.3 Scrum Roles 12

Conclusion **12**

Introduction

This chapter outlines our report and presents the framework of our project. It will introduce our host company, **Tunisiebooking**, our problem statement, the analysis of the existing system, and the proposed solution. We will also define our objectives and conclude with the adopted methodology.

1.1 Presentation of the Host Company

1.1.1 Overview and History

Tunisiebooking, operated by Link Tourism Partners Ltd, stands as a prominent travel agency specializing in hotel reservations across Tunisia. Established around 2014, the company has grown into a key player in the Tunisian tourism sector. Leveraging its online platform, Tunisiebooking connects travelers with over 500 partner hotels nationwide, establishing itself as a trusted name in the industry over the past decade.

TunisieBooking

Fig. 1.1. Tunisiebooking Official Logo

1.1.2 Operations and Scale

With a network of 34 agencies in Tunisia and a customer base exceeding 300,000 satisfied clients, Tunisiebooking has built its reputation on a user-friendly interface, competitive pricing, and reliable customer support available 7 days a week. Headquartered legally in Malta at The Core, Valley Road, Msida MSD 9021, the company maintains a strong operational presence in Tunisia, reflecting its deep ties to the local market. Employing an estimated workforce of 51 to 200, Tunisiebooking combines a mid-sized operational scale with a robust digital presence.

Fig. 1.2. Company statistics

1.1.3 Services and Mission

Tunisiebooking's services extend beyond hotel bookings to include flight comparisons (currently under development), organized trips, and specialized Omra packages, catering to a diverse clientele. The company's mission centers on delivering quality travel experiences at affordable rates. This is achieved through rigorous hotel selection, price negotiations, and weekly promotional offers, ensuring accessibility and value for its customers. Additionally, Tunisiebooking offers secure payment options and has plans for global hotel promotion expansions.

Aspect	**Details**
Legal Entity	Link Tourism Partners Ltd, Malta (Company No. C101832)
Operational Base	Houmt Souk, Avenue Hassine Anane, Djerba 4180, Tunisia
General Manager	Marwen Razgallah
Website	https://tn.tunisiebooking.com
Agencies	34 across Tunisia
Customer Base	Over 300,000 clients

Tab. 1.1. Tunisiebooking Company Profile

1.2 Project Presentation

The project focuses on the migration, redesign, and development of a modern, dynamic, and responsive hotel reservation website. It is specifically tailored for Algerian clients looking to travel to Tunisia and book hotels online with ease.

1.2.1 Problem Statement

The current TunisieBooking platform has several / technical and functional problems that affect how well it works and how users experience it.

Technically, it uses old backend technologies and a rigid database structure, making it hard to scale, update, or connect with modern tools.

Functionally, users can search for hotels and make simple bookings, but the platform lacks modern features like user accounts, booking history, personalized hotel suggestions, or loyalty programs.

The user interface is also outdated and not mobile-friendly, making it difficult to use on phones or tablets. These issues make it necessary to rebuild the platform using a modern, flexible architecture with better features, improved speed, and a more attractive design.

1.2.2 Study of the Existing System

This part looks at the current TunisieBooking website to understand what works and what needs to change.

Current System Overview

The current site was built using native PHP and serves as a hotel reservation site for Algerian users. It allows users to check availability and make simple bookings.

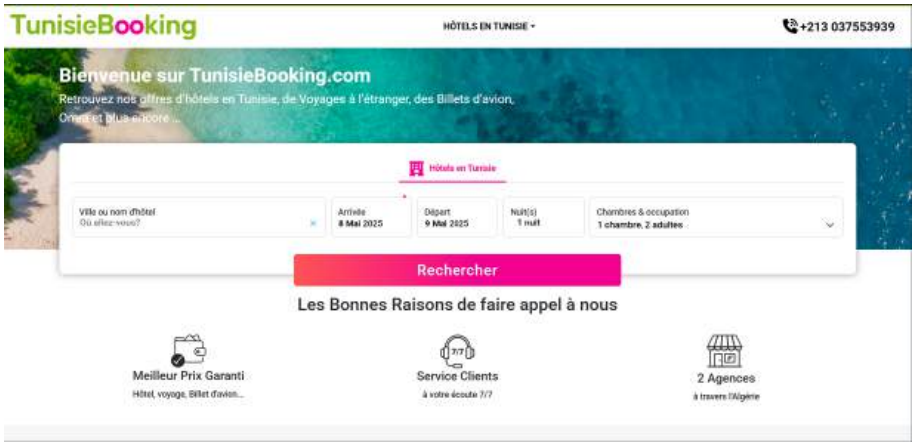


Fig. 1.3. Tunisiebooking Home Page

- **Main Limitations**

  The current version of the platform suffers from several functional and usability limitations. It lacks essential features that users now expect from modern booking systems. These gaps significantly impact the user experience and system effectiveness:

  - **No User Accounts**: Users can't register or manage bookings.
  - **Old Design**: The interface is not responsive and looks outdated.
  - **Few Features**: No AI tools, support chat, or personalized experience.

- **Technical Challenges**

  The system's outdated architecture limits the ability to add new features or improve existing ones. Technical debt and reliance on obsolete technologies make the platform harder to maintain and scale. These constraints slow down innovation and compromise performance:

  - **Old Code**: Built with native PHP, which makes updates hard.
  - **Security Issues**: Older PHP versions can have security flaws.

- **User Experience Comparison**

  Compared to leading platforms like Booking.com, this platform delivers a much less satisfying user experience. It lacks personalization, interactivity, and modern UI/UX standards. These deficiencies reduce user engagement and trust:

  - Personalized dashboards for users.
  - Smart search and hotel recommendations.
  - A clean, mobile-friendly design.

In short, the current platform lacks essential features, is built on outdated technology, and offers a poor user experience. These limitations highlight the need for a modern, secure, and user-friendly system that meets current user expectations and technological standards.

1.2.3 Proposed Solution

To address the current issues and enhance the experience for Algerian clients, we propose a complete redesign of the TunisieBooking platform using a modern system architecture. The new solution will leverage updated technologies to ensure better performance, scalability, and user satisfaction. On the backend, we will adopt the Laravel framework along with a MySQL database to provide a more secure and well-structured foundation. For the frontend, we will implement Next.js to deliver a responsive and modern user interface that ensures compatibility across all devices.

To highlight the key innovations of the new platform, we emphasize the following core functionalities:

- **User Features:** The platform offers tailored features for each user type. Visitors can search and book hotels. Clients access a personal dashboard to manage bookings, favorites, and get suggestions. Admins oversee hotel data and track platform activity.
- **AI Chatbot:** An AI-powered chatbot is available 24/7 to assist users with hotel searches, booking guidance, and instant support.

Main Improvements

Component	**Current System**	**New System**
Backend	Native PHP	Laravel Framework
Database	Old structure	Modern MySQL schema

Frontend	Static, not responsive	Modern and mobile-friendly (Next.js)
User Roles	Same for all users	Role-based features
Recommendations	Not available	AI Chatbot
Support	FAQ page only	Chatbot assistant

Tab. 1.2. Comparison between current and proposed systems

1.2.4 Objectives

The main objectives of this project are as follows:

- **Modernization**: Develop a modern, visually attractive, and fully responsive website that offers a smooth user experience across all devices.
- **Enhanced Functionality**: Introduce advanced features that extend beyond basic hotel reservations, including personalized experiences for registered users.
- **Technical Overhaul**: Design a new, scalable backend architecture with a robust database to replace the outdated logic of the existing system.
- **User Assistance and Engagement**: Integrate a smart chatbot capable of guiding users in selecting suitable hotels and providing real-time assistance throughout the booking process.
- **Scalability and Administrative Control**: Build a platform that facilitates efficient content management and user administration, supporting future growth and operational ease.

1.3 Development Methodology

Given the collaborative nature and evolving requirements of the TunisieBooking platform, selecting an appropriate development methodology was essential. We adopted an Agile approach to ensure flexibility, continuous delivery, and effective communication among team members. After evaluating several methodologies, we chose the Scrum framework due to its structured yet adaptable nature and its proven effectiveness in web application projects.

1.3.1 Evaluation of Methodologies

Selecting the right methodology helps manage project complexity, supports scalability, and facilitates timely delivery. We considered the following options:

Waterfall

The Waterfall model is a linear, sequential approach in which each phase requirements, design, implementation, testing, and deployment must be completed before the next begins. Although effective for projects with fixed, well-defined requirements, its lack of adaptability makes it unsuitable for TunisieBooking, where features may evolve based on user feedback.

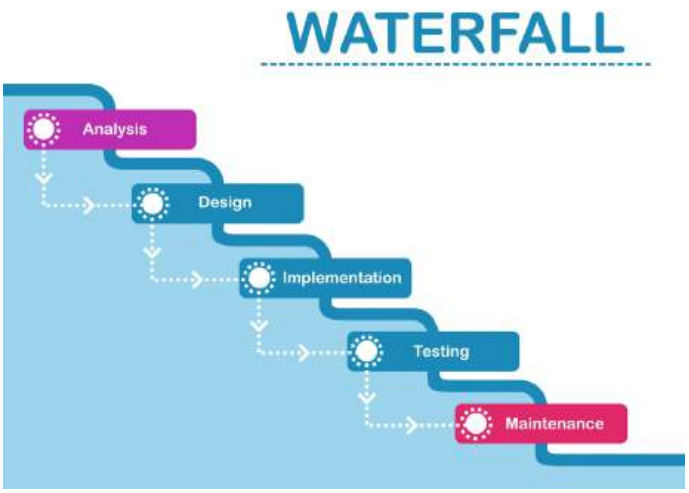


Fig. 1.4. Waterfall Methodology Process

Scrum (Agile)

Scrum is a widely adopted Agile framework that structures the development process into fixed-length iterations known as *sprints*, typically lasting two weeks. It emphasizes collaboration, transparency, and continuous improvement, making it well-suited for projects with evolving requirements. By fostering frequent inspection and adaptation, Scrum enables teams to respond quickly to changes and deliver value incrementally.

- **Daily Stand-ups**: Brief, time-boxed meetings where team members share progress, identify roadblocks, and synchronize their efforts to maintain momentum.
- **Sprint Retrospectives**: Conducted at the end of each sprint to reflect on what went well, what can be improved, and how the team can enhance its workflow in future sprints.
- **Sprint Planning and Reviews**: Each sprint begins with a planning session to define objectives and ends with a review to demonstrate completed work and gather stakeholder feedback.

Scrum's iterative approach and its strong focus on communication and feedback loops make it particularly effective for dynamic, user-centric platforms like *TunisieBooking*, where user

needs and priorities can shift rapidly. Its structure encourages early delivery of working features, risk reduction, and better alignment with business goals.

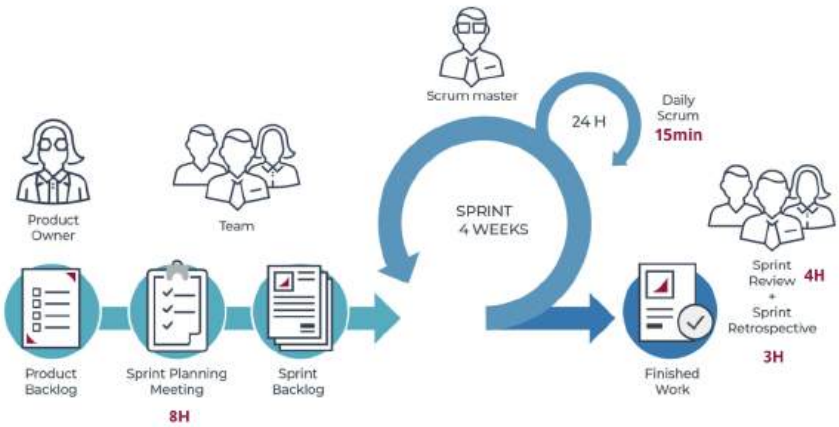


Fig. 1.5. Scrum Methodology Process

Kanban (Agile)

Kanban emphasizes task visualization and continuous delivery using boards with columns (e.g., To Do, In Progress, Done). While it offers flexibility and transparency, the absence of time-boxed planning makes it less effective for managing release milestones.

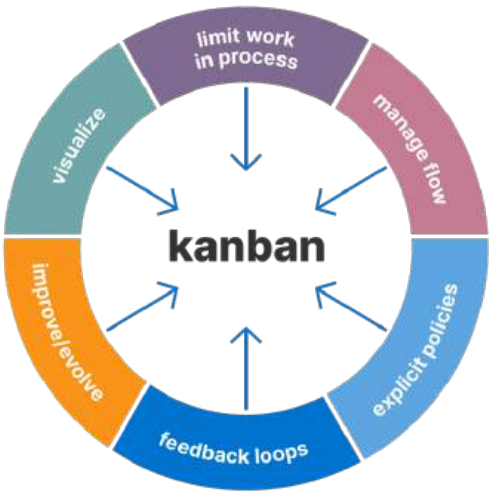


Fig. 1.6. Kanban Methodology Process

2TUP (Two-Track Unified Process)

2TUP blends the planning rigor of Waterfall with Agile's iterative cycles, splitting development into two parallel tracks:

- **Functional Track**: Focused on requirement analysis and system design.
- **Technical Track**: Dedicated to implementation, testing, and deployment.

While methodical, 2TUP's complexity and slower cycles make it less responsive to evolving user needs.

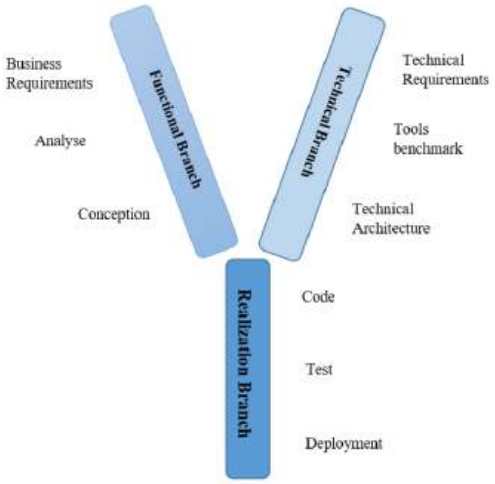


Fig. 1.7. 2TUP Methodology Process

1.3.2 Chosen Methodology: Scrum

Compared to the Waterfall model, Scrum offers greater adaptability and responsiveness to change. While Kanban emphasizes continuous delivery, it lacks the time-boxed structure that Scrum provides for better planning and sprint goals. Unlike 2TUP (Two-Track Unified Process), Scrum reduces the overhead of heavy documentation and long upfront planning, allowing for shorter feedback loops and quicker decision-making.

Scrum offers the following benefits:

- Iterative development and frequent feature releases.
- Flexibility to incorporate changing user requirements.
- Clearly defined roles and responsibilities.
- Enhanced communication through daily meetings and sprint reviews.

Compared to Waterfall, Scrum supports adaptability. Unlike Kanban, it introduces structure and prioritization. And in contrast to 2TUP, it allows for faster cycles and reduced planning overhead.

1.3.3 Scrum Roles

Scrum defines three key roles that promote accountability and collaboration:

1. **Product Owner**
   Represents the client's vision, defines the product backlog, and ensures that development efforts align with business goals.

2. **Scrum Master**
   Facilitates the Scrum process, removes impediments, and ensures adherence to Agile principles.

3. **Development Team**
   A cross-functional group responsible for delivering working product increments at the end of each sprint.

The roles within the TunisieBooking team are as follows:

Scrum Role	Assigned Member
Product Owner	Zagrouba Lazhar
Scrum Master	Jouili Imed
Development Team	Rayen Ben Hassan

Tab. 1.3. Scrum Team Assignments

Conclusion

This chapter laid the foundation for the TunisieBooking project by introducing the host company, identifying system shortcomings, presenting a proposed solution, and setting clear objectives. After comparing various development methodologies, we adopted Scrum to ensure iterative progress, adaptability, and efficient collaboration. This methodological foundation will guide the upcoming phases of design and implementation.

Chapter 2

Sprint 0: General project planning

Contents

Introduction **13**

2.1 Requirements Study **14**

2.1.1 Identification of Actors 14

2.1.2 Functional Requirements 14

2.1.3 Non-Functional Requirements 15

2.2 Project Management with SCRUM **16**

2.2.1 Product Backlog 16

2.2.2 Sprint Planning 18

2.2.3 General Use Case Diagram 19

2.2.4 General Class Diagram 20

2.3 Technical Choices **21**

2.4 Application Architecture **24**

2.4.1 Logical Architecture 24

2.4.2 Physical Architecture 25

Conclusion **27**

Introduction

This chapter presents the functional analysis of the system, starting with user profiles and key platform features. It then outlines the chosen technical architecture, tools, and the use of Scrum to guide the development process.

2.1 Requirements Study

This section provides a comprehensive overview of the requirements for the TunisieBooking platform. It includes both functional requirements describing the features and behaviors expected by users and non functional requirements, which define performance, scalability, usability, and other system constraints.

2.1.1 Identification of Actors

In software engineering, an actor is any external entity (user or system) that interacts with the application. For TunisieBooking, we have identified the following main actors:

Actor	Description
Super Admin	Manages all system functionalities.
admin	Handles some of system features.
Visitor	Unregistered user browsing the site.
Client	User with a created account.

Tab. 2.1. User Roles Overview

2.1.2 Functional Requirements

Functional requirements describe what each user can do on the platform, depending on their role. Shared functionalities are grouped under main roles, with additional permissions shown for extended roles.

1. admin:

- **Authentication:** Login and logout securely.
- **User Management:** Add, edit, delete users; assign roles; search and filter users.
- **Agency Management:** Create, update, delete agencies; activate or deactivate them; search agencies.
- **Reservation Management:** Create, view, modify, or cancel bookings; compare hotel offers; assist clients; print invoices.
- **Hotel Management:** Edit hotel details and themes.
- **Hotel Chains & Themes:** View and search chains and themes.
- **Analytics Dashboard:** View statistics on hotels, bookings, and revenue.

2. Super admin: Has all admin permissions, plus the ability to:

- **Supplier Management:** Add, edit, delete, or view API suppliers; configure API keys; manage exchange rates and profit margins; activate/deactivate suppliers.
- **Hotel Mapping:** Identify and merge duplicate hotel listings from different suppliers.

3. Visitor:

- **Hotel Browsing:** View and filter hotel listings (by location, date, price, etc.).
- **Offers and Information:** Access general content and promotions.
- **Booking Without Account:** Reserve hotels using personal details (no login required).
- **Contact Agencies:** Reach out for help or questions.
- **Account Registration/Login:** Create or access an account for extra features.

4. Client: Has all Visitor features, with access to:

- **Account Dashboard:** Manage profile and preferences.
- **Booking History:** View and cancel previous reservations (depending on policy).
- **Favorite Hotels:** Save and view preferred accommodations.
- **AI Chatbot Access:** Get suggestions and instant help via the integrated chatbot.

2.1.3 Non-Functional Requirements

The non-functional requirements define the quality attributes, operational constraints, and implementation standards that the system must meet to ensure optimal performance, reliability, and user experience.

- **Performance:** The system must support a high volume of concurrent users without performance degradation. Page load times should not exceed 3 seconds under normal conditions.
- **Scalability:** The platform should be scalable to handle increased user traffic and an expanding database of hotel listings. System updates and scaling operations must be performed with minimal downtime.

- **Reliability:** The system should maintain at least 99% uptime to ensure consistent service availability to users.
- **Security:** All sensitive user data, including personal must be protected using industry standard encryption protocols. The system must comply with applicable data protection regulations.
- **Compatibility:** The application must be compatible with all major web browsers (e.g., Chrome, Firefox, Safari) and provide a responsive user interface that functions smoothly across desktop, tablet, and mobile devices.
- **Availability:** The platform should be accessible 24/7. Planned maintenance should be announced in advance and limited to a maximum of 2 hours per month.

These non-functional requirements aim to ensure that **TunisieBooking** delivers a secure, stable, and seamless experience for all users, while remaining adaptable to future growth and technological advancements.

2.2 Project Management with SCRUM

SCRUM is an agile project management methodology designed to enhance team collaboration, adaptability to change, and timely delivery of high-quality products. It emphasizes iterative development, frequent communication, and continuous feedback to ensure the project evolves in alignment with stakeholder expectations.

2.2.1 Product Backlog

The Product Backlog is a key SCRUM artifact listing all features, improvements, and fixes, prioritized by business value and complexity.

Items are categorized as:

- **High**: Essential and urgent features.
- **Medium**: Useful enhancements with moderate priority.
- **Low**: Non-urgent features for future consideration.

Example of Prioritized User Stories

ID	User Story	Description	Priority
1	Authentication	As a Super Admin / Admin, I want to log in and log out securely to access platform functionalities.	High
2	Supplier Management	As a Super Admin, I want to manage suppliers, configure API keys, set profit margins, and manage currency settings.	High
3	Hotel Mapping	As a Super Admin, I want to unify duplicate hotel listings across different suppliers.	High
4	User Management	As a Super Admin / Admin, I want to manage users, assign roles, and filter/search through user records.	High
5	Agency Management	As a Super Admin / Admin, I want to manage travel agencies and control their status (active/inactive).	High
6	Reservation Management	As a Super Admin / Admin, I want to book, edit, or cancel reservations and compare offers from suppliers.	High
7	Hotel Content Management	As a Super Admin / Admin, I want to update hotel descriptions and assign themes to listings.	Medium
8	Analytics	As a Super Admin / Admin, I want to view platform statistics and charts related to hotels, reservations, and revenues.	Medium
9	Authentication	As a Visitor / Client, I want to register, log in, and log out securely to access personalized features.	High
10	Browse and Search Hotels	As a Visitor / Client, I want to browse hotel listings and use filters to find suitable options.	High
11	View Offers	As a Visitor / Client, I want to view static and promotional offers to plan my trips better.	Medium
12	Start Booking	As a Visitor / Client, I want to initiate a reservation and be prompted to log in or register to complete it.	High

13	Booking Management	As a Client, I want to view, modify, or cancel my reservations according to platform policy.	High
14	Personalized Suggestions	As a Client, I want to receive hotel recommendations based on my preferences and past activity.	Medium
15	Secure Booking	As a Client, I want to book hotels through a secure system and receive confirmation.	High
16	Favorites	As a Client, I want to save and view a list of favorite hotels.	Medium
17	Contact Agencies	As a Visitor / Client, I want to contact agencies for inquiries or assistance.	Medium
18	Email Notifications	As a Client, I want to receive booking confirmations and updates via email.	Medium
19	Chatbot Usage	As a Client, I want to use a chatbot to guide me through the application and recommend what suits my needs.	Medium

Tab. 2.2. Product Backlog

2.2.2 Sprint Planning

The following table outlines the sprint planning for Release 1 and 2 of the TunisieBooking project, detailing the tasks to be completed in each sprint.

Sprints	**Tasks To Do**
Sprint 1 (4 weeks)	• Authentication development • Password reset and email verification • Security implementation (csrf) • User Managment

Sprint 2 (4 weeks)	• Setup hotels availability api's • Develop the hotel searching functionality • Develop the hotel booking system
Sprint 3 (4 weeks)	• Setup admin dashboard environement • Setup super admin dashboard environement • Setup hotels mapping logic
Sprint 4 (4 weeks)	• Setup client space • AI integration • Configure favorite hotels logic

Tab. 2.3. Sprint Planning

2.2.3 General Use Case Diagram

A use case diagram models the interactions between a system and its users. It consists of:

- **System**: Represents the scope of the study.
- **Actor**: Represents people or external systems interacting with the system.
- **Use Case**: Represents the functionalities offered by the system.
- **Association**: The links between actors and use cases, or between use cases themselves.
- **Generalization**: A hierarchical relationship between actors (an actor may inherit the characteristics of another).
- **Others**: Additional elements such as includes, extends, etc.

The use case diagram presented in Fig. 3.1 provides a graphical representation of the application's functional requirements. It models the interactions between actors and use cases, as well as the relationships between the various use cases.

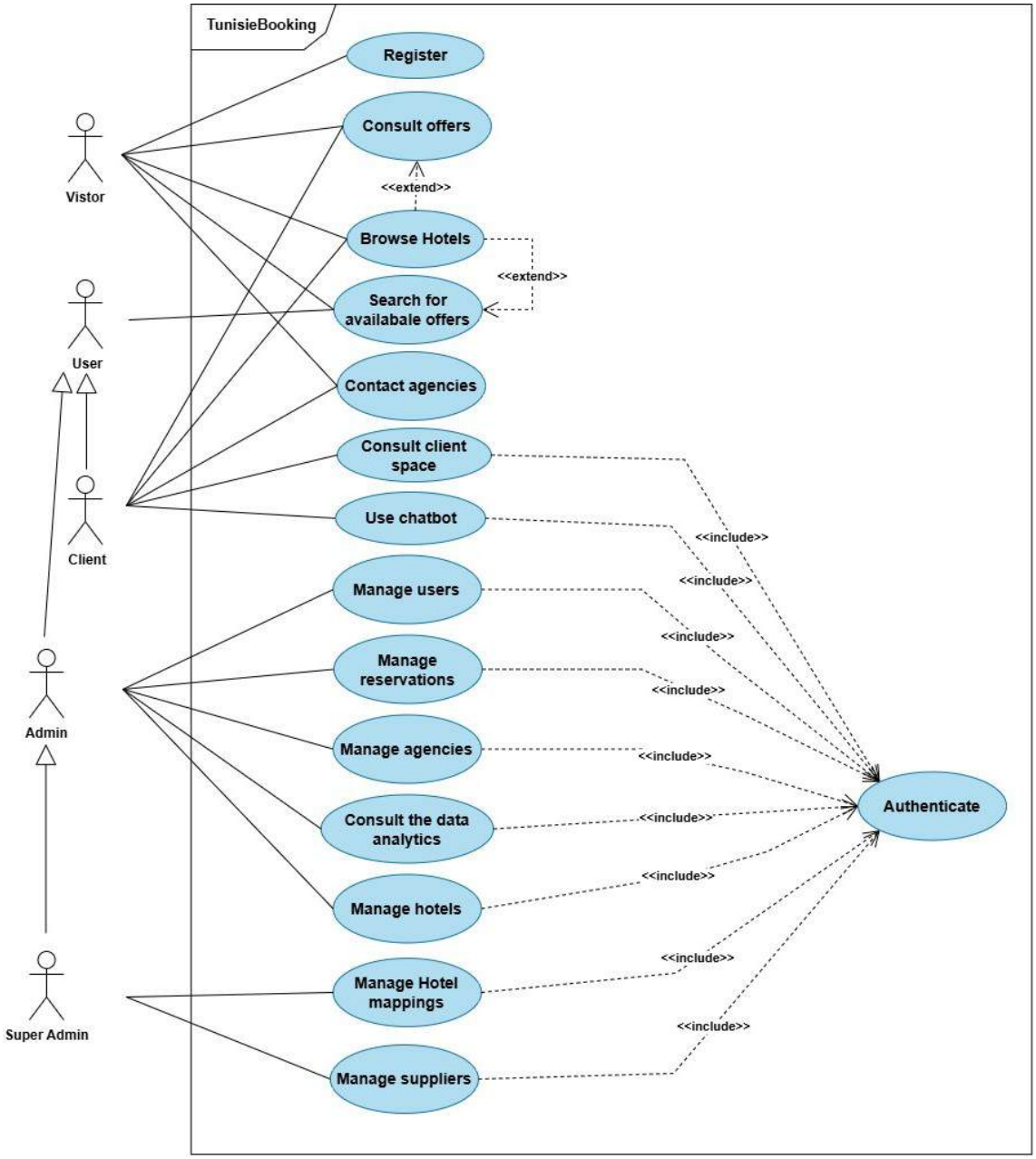


Fig. 2.1. General Use Case Diagram

2.2.4 General Class Diagram

The general class diagram illustrates the application's static structure by presenting its classes, their attributes, and the relationships between them.

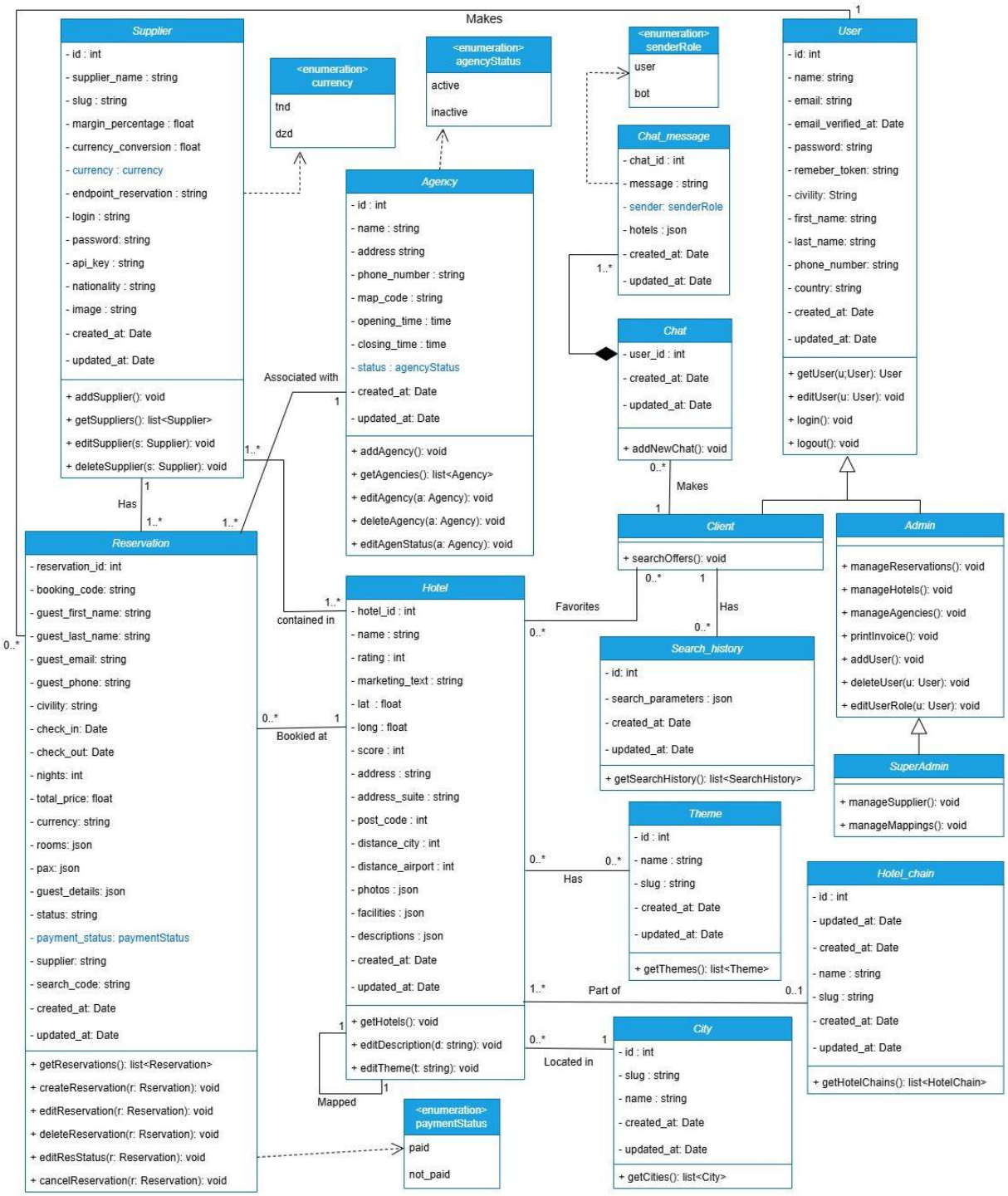


Fig. 2.2. General Class Diagram

2.3 Technical Choices

The technical foundation of a web project, though often invisible to end-users, is critical to the application's success. It defines how the platform operates, scales, and evolves. For **TunisieBook-**

ing, the technical architecture is divided into two main components: the **backend** and the **frontend**.

Backend Development

The backend is responsible for handling business logic, data management, and server-side operations. It is composed of two primary elements:

- **Database:** TunisieBooking uses **MySQL**, a powerful and widely adopted relational database management system written in C and C++. MySQL is ideal for managing structured data, such as hotel information, user profiles, and reservation records. It ensures data consistency through predefined schemas and supports scalability across distributed systems.

Fig. 2.3. MySQL Logo

- **Server Framework:** The server is responsible for processing requests, executing business logic, and facilitating communication between the database and the frontend.

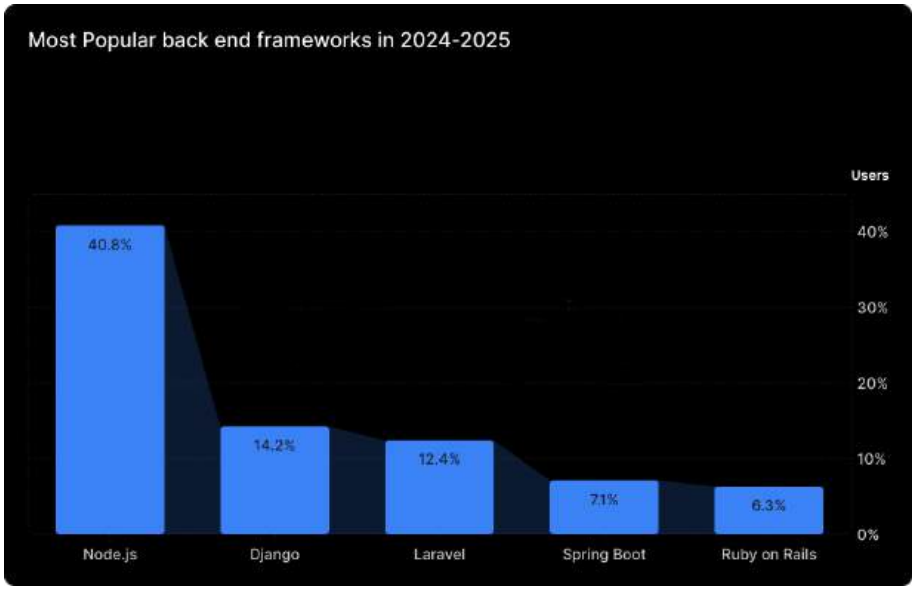


Fig. 2.4. Ranking of backend frameworks in 2024-2025

Selected Technology: We opted for **Laravel PHP**, a robust and developer-friendly open-source framework. Laravel (version 8.x or higher) enables the development of scalable web applications with features such as Eloquent ORM for database interaction, built-in authentication, middleware, and RESTful API support. Laravel runs on web servers like Apache or Nginx and uses Composer for dependency management.

Fig. 2.5. Laravel Logo

Frontend Development

The frontend represents the user interface and is responsible for everything the user sees and interacts with. For TunisieBooking, the goal is to deliver a fast, responsive, and user-friendly experience.

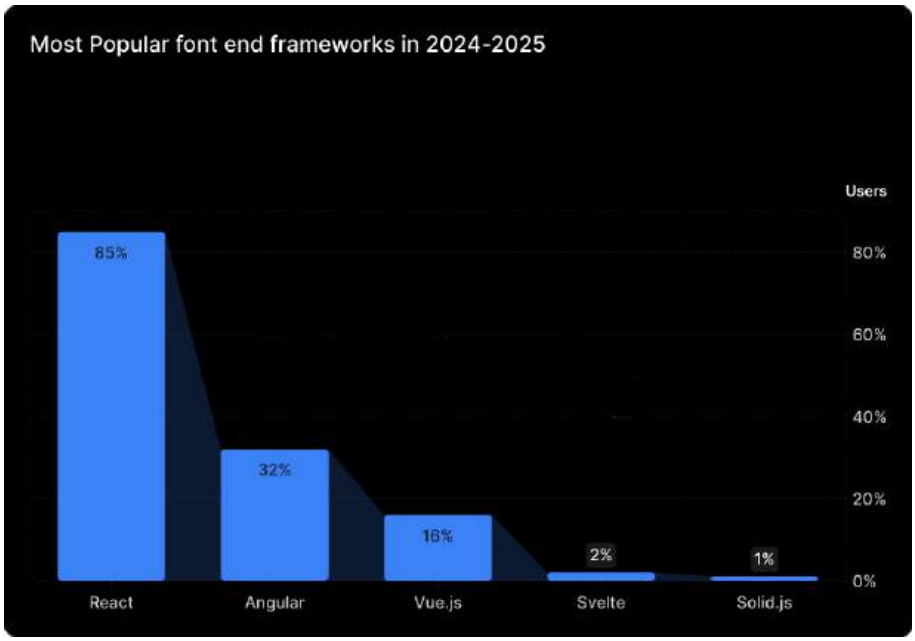


Fig. 2.6. Ranking of frontend frameworks in 2024-2025

Selected Technology: We chose **Next.js**, a React based JavaScript framework that supports both server side rendering and static site generation. It also offers built-in routing, excellent

performance, and SEO optimization. Next.js requires Node.js (version 16.x or higher) and supports npm or Yarn for package management. It is well suited for building dynamic, scalable web applications.

Fig. 2.7. Next.js Logo

Styling: For UI styling, we integrated **Tailwind CSS**, a utility-first CSS framework that complements Next.js. Tailwind enables rapid development of responsive and consistent designs without the need for writing large amounts of custom CSS.

Fig. 2.8. Tailwind CSS Logo

Conclusion: By combining **Laravel PHP** for the backend with **Next.js** and **Tailwind CSS** for the frontend, TunisieBooking benefits from a modern, efficient, and scalable technology stack. This stack ensures high performance, maintainability, and a seamless user experience across all devices.

2.4 Application Architecture

2.4.1 Logical Architecture

Developing a robust and scalable platform like **TunisieBooking** requires a solid architectural foundation. To ensure modularity, maintainability, and separation of concerns, the application adopts the well-established **Model-View-Controller (MVC)** architectural pattern.

Model-View-Controller (MVC)

The MVC pattern divides the application into three interconnected components:

- **Model:** Represents the data and business logic of the application. It defines how data is structured, retrieved, and manipulated. The Model enforces business rules and handles the state of the application.
- **View:** Handles the presentation layer. It is responsible for displaying the data received from the Controller. The View observes the Model and updates the UI automatically when the data changes, following the Observer design pattern.
- **Controller:** Acts as an intermediary between the View and the Model. It handles user input, processes it (often via the Model), and returns the appropriate response to the View.

The MVC architecture offers several advantages. It supports multiple views of the same data model, encourages asynchronous communication between components, and enhances modularity.

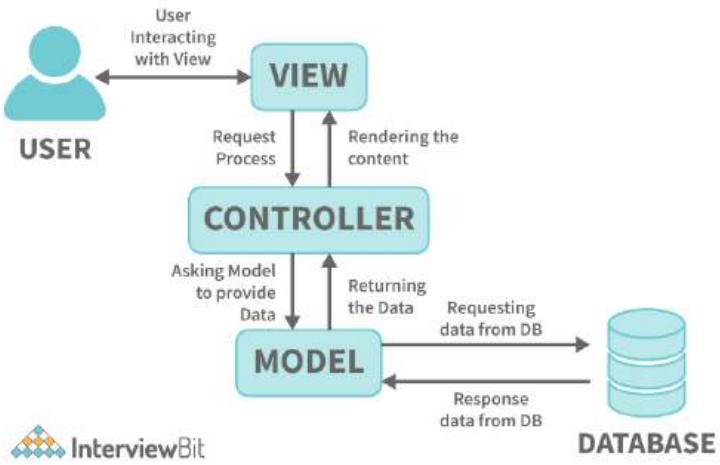


Fig. 2.9. MVC architecture.

2.4.2 Physical Architecture

TunisieBooking adopts a **3-Tier Architecture**, a client-server model that separates the user interface, application logic, and data storage into distinct layers. This architectural choice enhances modularity, improves system scalability, and facilitates independent deployment and maintenance of each tier.

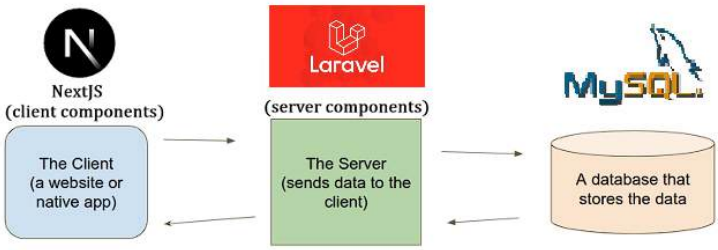


Fig. 2.10. Physical architecture.

Presentation Tier (Client Tier)

The presentation tier is the front-facing layer responsible for rendering the user interface and managing user interactions.

- **Components:** Web browsers, mobile devices.
- **Technologies:** Next.js, Tailwind CSS.
- **Role:** Uses Next.js to handle server-side rendering and client-side interactivity. Tailwind CSS is used to style the interface, ensuring responsiveness across all devices. This tier sends user requests (e.g., search queries) to the application tier and displays returned results.

Application Tier (Server Tier)

The application tier is responsible for processing business logic and orchestrating communication between the frontend and the database.

- **Components:** Application servers.
- **Technologies:** Laravel PHP.
- **Role:** Handles API endpoints, processes business rules (e.g., booking validation, currency conversion), and serves responses to the frontend. It ensures secure and efficient logic handling using Laravel's features like middleware, routing, and authentication.

Data Tier (Database Tier)

This tier manages persistent data storage, handling all database operations including data retrieval, insertion, and updates.

- **Components:** Database servers.

- **Technologies:** MySQL.
- **Role:** Stores and manages structured data including hotel listings, user accounts, booking history, and supplier configurations. It ensures data integrity and security through relational constraints and user permissions.

Advantages of 3-Tier Architecture

- **Scalability:** Each layer can be scaled independently based on demand, improving performance without requiring a full system overhaul.
- **Maintainability:** Changes in one tier (e.g., UI enhancements or logic updates) do not directly affect others, simplifying debugging and future development.
- **Reusability:** The application and data tiers can be reused across other platforms or projects (e.g., mobile apps or admin portals).
- **Security:** Each tier can enforce its own security protocols, minimizing attack vectors and ensuring better protection of sensitive data.

Physical Deployment

The system is containerized using Docker for modularity and scalability:

- **Client Tier:** Runs in a container serving the front-end, accessible via browsers and mobile devices.
- **Application Tier:** Laravel back-end hosted in a container, deployed on our VPS from OVH Cloud.
- **Data Tier:** MySQL database runs in a separate container with persistent storage and optional clustering.

Conclusion

This chapter outlined the project scope, key stakeholders, system architecture, and development plan. With this foundation, we are ready to begin implementation in Sprint 1.

Chapter 3

Study and implementation of sprint 1

Contents

Introduction **28**

3.1 Sprint 1 Objective **28**

3.2 Sprint 1 Backlog **29**

3.3 Design **29**

3.3.1 Sprint 1 Use Case Diagram 30

3.3.2 Class diagram 32

3.3.3 Sequence diagram 33

3.4 Implementation **33**

Conclusion **37**

Introduction

Following the needs analysis, we are starting the development phase by focusing on the elements of the first sprint. The first sprint is dedicated to setting up the foundations of the authentication system and user management.

3.1 Sprint 1 Objective

The goal of Sprint 1 is to build a secure authentication system with features like registration, login, password reset, email verification, CSRF protection, and user management for administrators.

3.2 Sprint 1 Backlog

User Story	**Tasks**	**Estimate (Days)**
As a visitor, I want to register so that I can have an account.	• Design forms • Frontend + backend logic • API connection • Testing	4
As a visitor/admin/super admin, I want to login in securely so that I can access to dashboards.	• Design forms • Frontend + backend logic • API connection • Enable CSRF in Laravel • Add tokens to forms • Test protection	3
As a client, I want to reset my password.	• UI for reset/verify • Token logic • Email service • Tests	2
As a client, I want to verify my email so that I can recover access and ensure my account is secure.	• UI for reset/verify • Token logic • Email service • Tests	3
As an admin, I want to manage users so that I can control access, roles, and view client activity efficiently.	• UI for CRUD • Backend logic • Role handling • Search/filter • Testing	13

Tab. 3.1. Sprint 1 Backlog

3.3 Design

Diagrams are grouped into two main families: structural diagrams and dynamic diagrams. Structural diagrams describe the static structure of the system. They include class diagrams, object diagrams, component diagrams, deployment diagrams, package diagrams and composite structure diagrams. Dynamic diagrams, on the other hand, describe the system's behavior over time, including interactions between elements and activity flows. This category includes use

case diagrams, sequence diagrams, as well as communication, activity, state (or state-transition), timing and global interaction diagrams.

3.3.1 Sprint 1 Use Case Diagram

The following use case diagram illustrates the key functionalities of the authentication and user management system developed during Sprint 1.

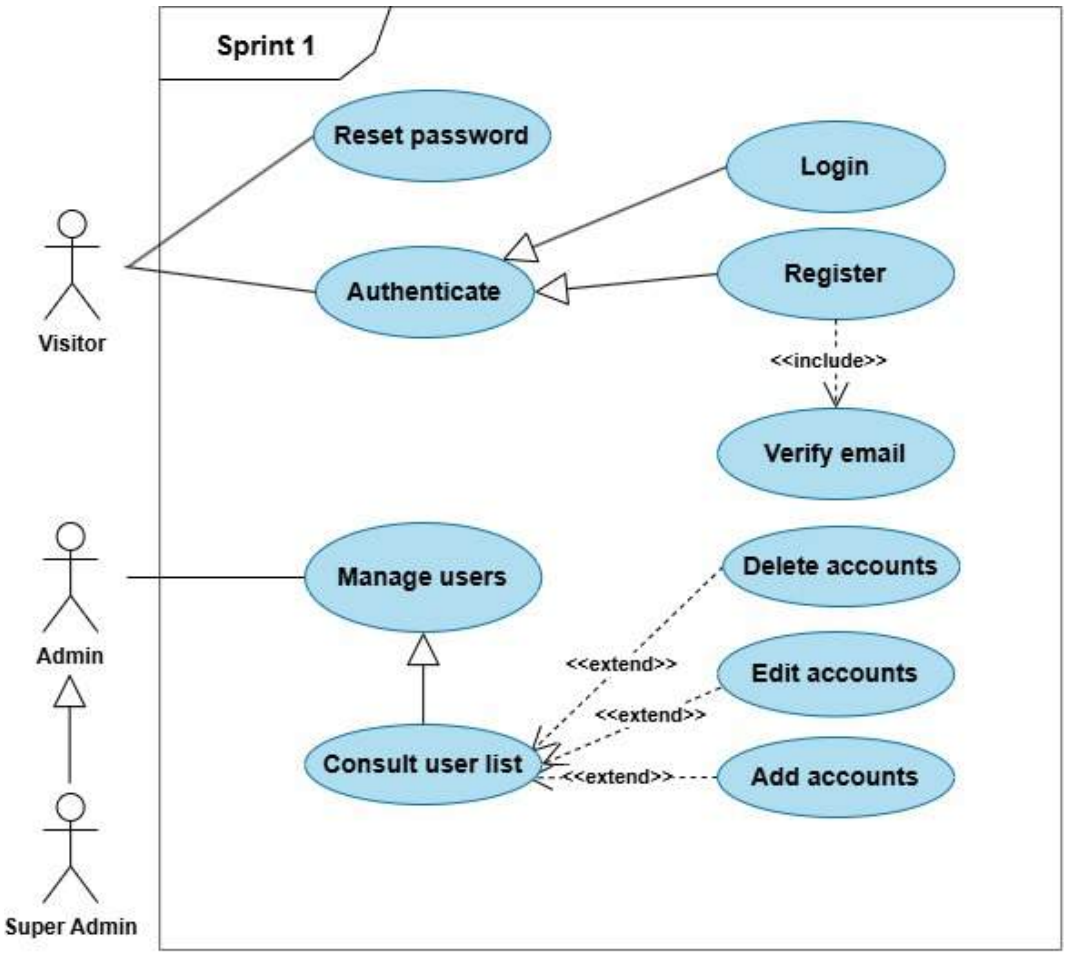


Fig. 3.1. Use Case Diagram – Authentication and User Management (Sprint 1)

Textual Description of the Use Case "Authentication"

Title	**Authenticate**
Actor	Visitor
Pre-condition	The user must have an account or will register an account.
Post-condition	The user can access the client space, chatBot and more features.

Normal Scenario	1. The system displays the ”Authentication” interface. 2. The user fills in the fields necessary for authentication. 3. The user clicks on the ”Authenticate” button. 4. The system verifies the provided information. 5. The system approves a verification code for security generated by the token. 6. The system redirect the user to the home page
Exception Scenario	A2. **Incorrect username or password**: The system displays an error message. A3. **User does not exist in the database**: The system displays a login failure message.

Tab. 3.2. textual description of the use case Authentication

Textual Description of the Use Case “Reset Password”

Title	Reset Password
Actor	Client / Admin / Super Admin
Pre-condition	The user has an existing, verified account.
Post-condition	The user sets a new password and can log in.
Main Scenario	1. User clicks “Forgot Password”. 2. System displays the reset-request form. 3. User enters registered e-mail and submits. 4. System sends a reset link e-mail. 5. User opens the link, enters a new password, and confirms. 6. System updates the password and shows a success message.
Exception Scenarios	B1. **E-mail not found:** System shows “Account not found”. B2. **Expired token:** System asks the user to request a new link.

Tab. 3.3. Textual description of the use case “Reset Password”.

Textual Description of the Use Case “Manage Users”

Title	Manage Users
Actor	Admin / Super Admin
Pre-condition	Admin is logged in with the required permissions.

Post-condition	User list is updated (accounts added, edited, or removed).
Main Scenario	1. Admin opens the “Manage Users” page. 2. Admin chooses *Add*, *Edit*, or *Delete*. 3. For *Add*: admin enters new user details and submits. 4. For *Edit*: admin modifies selected user data and saves. 5. For *Delete*: admin confirms removal. 6. System applies the change and refreshes the list.
Exception Scenarios	C1. **Invalid data:** System shows an error (e.g., duplicate e-mail). C2. **User not found:** System displays “User does not exist”. C3. **Permission denied:** System blocks the action.

Tab. 3.4. Textual description of the use case “Manage Users”.

3.3.2 Class diagram

The class diagram shows a user hierarchy for an authentication system, where User is the base class, and Client, Admin, and SuperAdmin inherit from it with role-specific features.

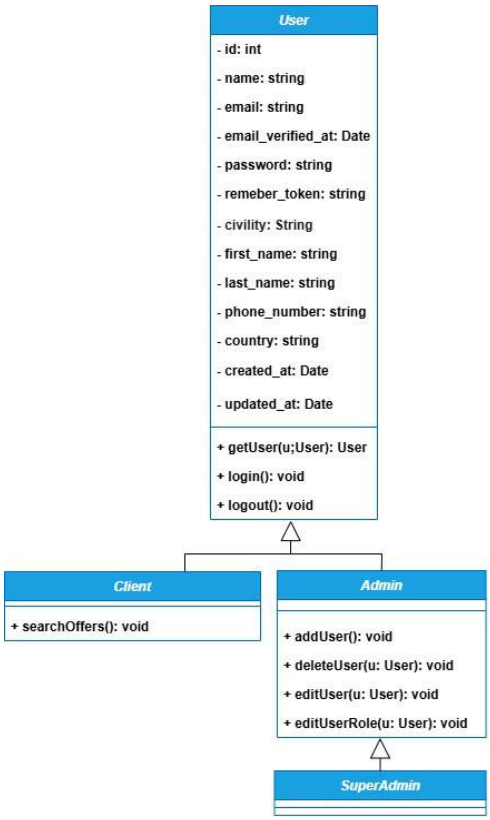


Fig. 3.2. Class diagram – Authentication and User Management (Sprint 1)

3.3.3 Sequence diagram

This diagram shows the step-by-step flow of the authentication process. It begins when the user submits their credentials through the interface. The credentials, along with a CSRF token, are sent to the authentication controller. The controller verifies the credentials by querying the database. Depending on the result, the system either grants access with a session ID and redirects the user to the home page, or displays an error message and redirects them back to the login page.

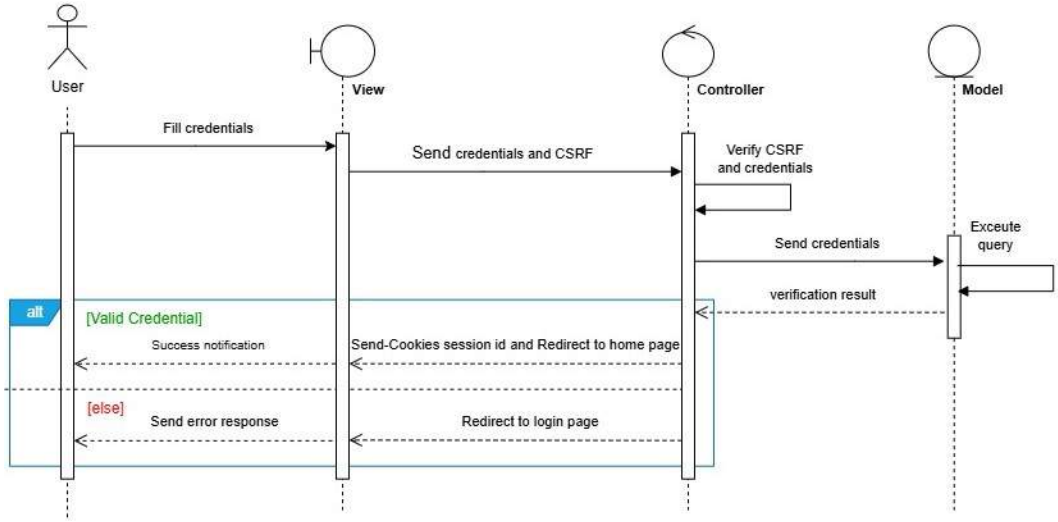
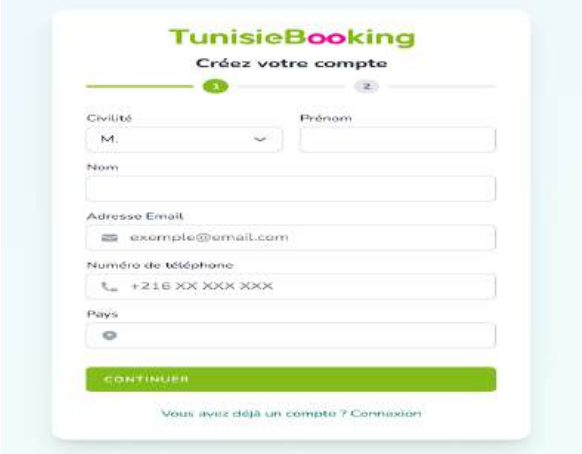


Fig. 3.3. Authentication Sequence diagram

3.4 Implementation

This section presents the various user interfaces developed during Sprint 1. These interfaces cover both the authentication flow including registration, login, email verification, and password reset as well as user management functionalities such as adding, editing, and deleting users. Together, they ensure a secure, efficient, and user-friendly experience for both clients and administrators.

Registration Page: This page allows new users to create an account by providing their name, email, password, and other required information.



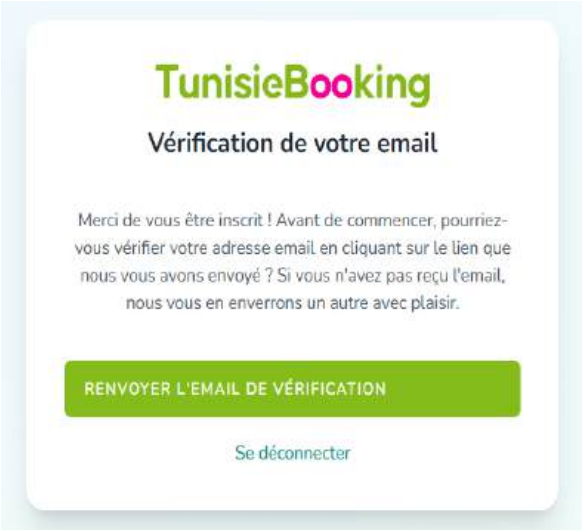
(a) Registration Interface (Step 1)

(b) Registration Interface (Step 2)

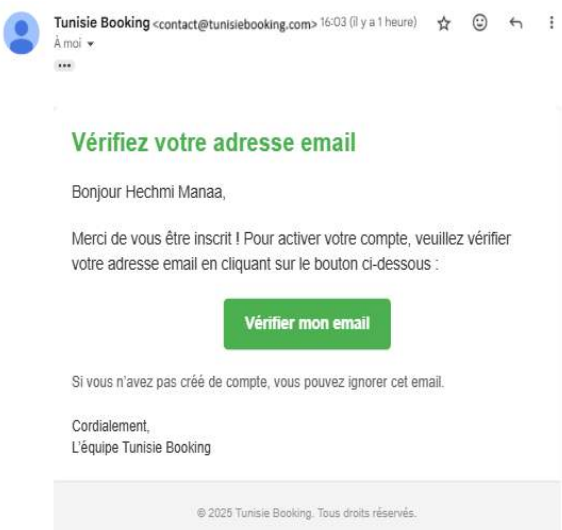
Fig. 3.4. Registration Interfaces

It includes input validation to ensure data integrity and initiates the email verification process upon successful registration.

Email Verification: After registration, the user receives a verification email (Fig. 3.5b). Clicking the verification link redirects the user to a confirmation screen (Fig. 3.5a), completing the account activation.



(a) Email Verification Screen

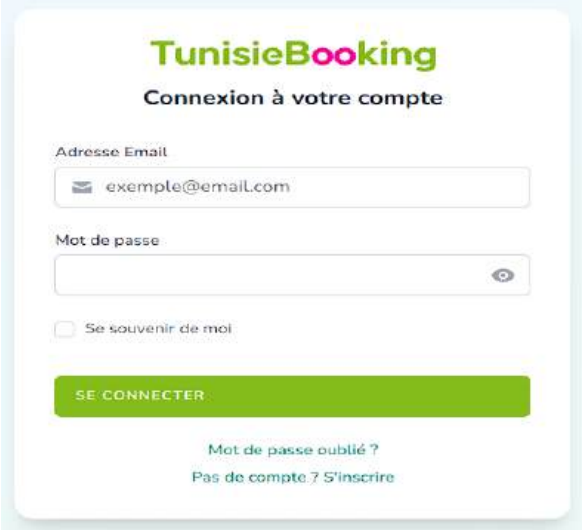


(b) Verification Email

Fig. 3.5. Email Verification Interfaces

Login Interface: This page allows users to access the system by submitting their login credentials. It handles authentication errors such as incorrect email or password (a).

Password Reset Request: Users who forget their password can initiate the reset process by entering their registered email address. A reset link is then sent to their inbox (b).

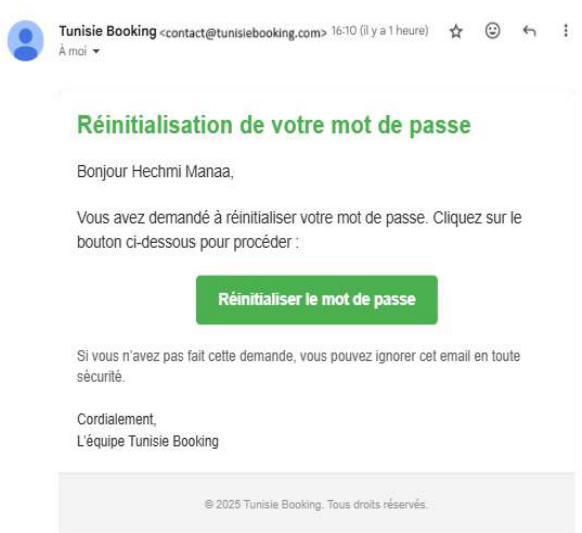


(a) Login Interface

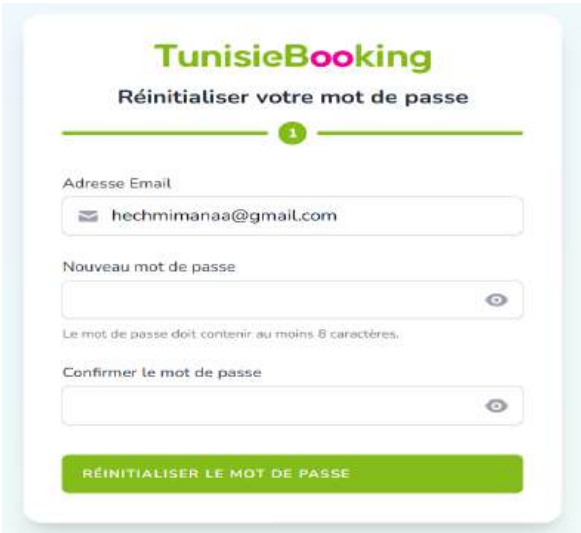
(b) Password Reset Request Page

Fig. 3.6. Login and Password Reset Request Interfaces

Password Reset Completion: After clicking the reset link received by email (Fig. 3.7a), the user is redirected to a form (Fig. 3.7b) where they can define a new password to complete the reset process.



(a) Password Reset Email



(b) New Password Form

Fig. 3.7. Password Reset Completion Interfaces

User Management Interface: This dashboard displays a list of all registered users, along with key actions available for each: viewing user details, editing user information, deleting the user, or changing their role. It also provides useful filters such as search by name, email, and date of registration. The total number of users is displayed at the top to give administrators a

quick overview.

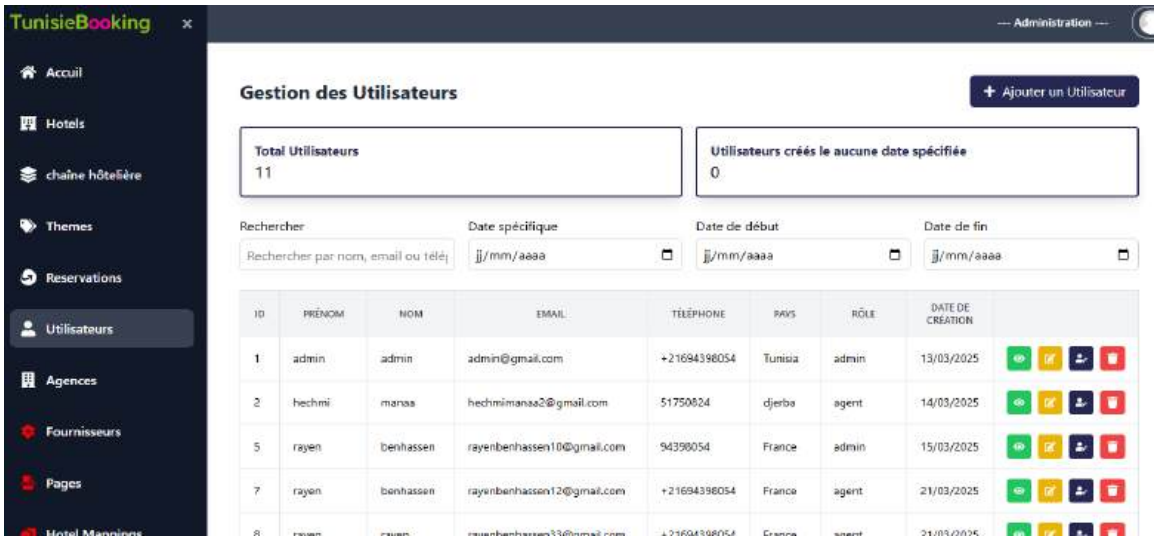
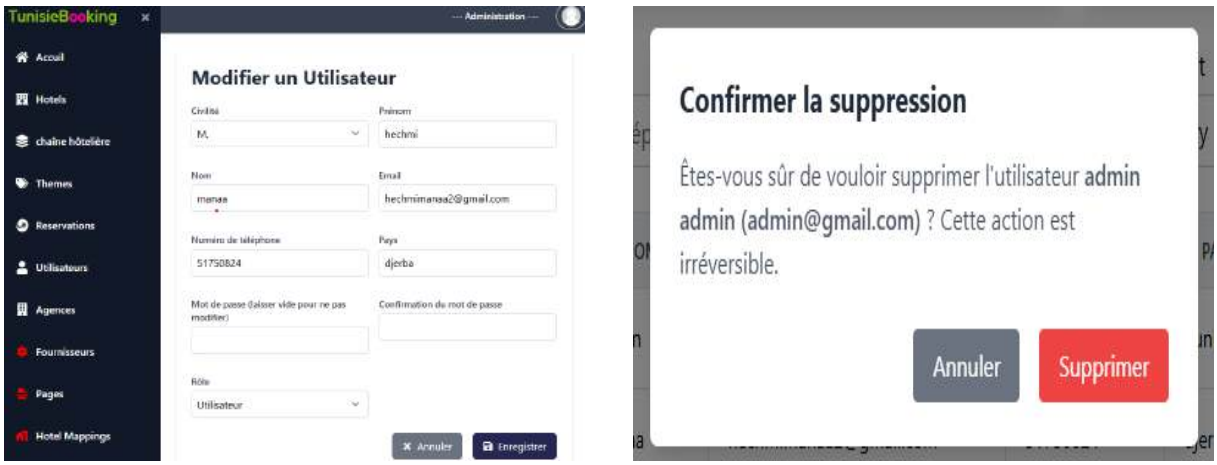


Fig. 3.8. User Management Dashboard

Edit User Interface: This interface allows administrators to modify user details such as name, email, and role. It ensures all updates are validated before submission.

Delete User Confirmation: When an admin chooses to delete a user, a confirmation message is displayed to prevent accidental deletion. This step adds an extra layer of safety to user management.



(a) Edit User Interface

(b) Delete Confirmation Dialog

Fig. 3.9. User Editing and Deletion Interfaces

Add User Interface: This page provides a form for administrators to manually add new users to the system. It includes fields such as name, email, password, and role selection. Input validation ensures the new user data meets system requirements before creation.

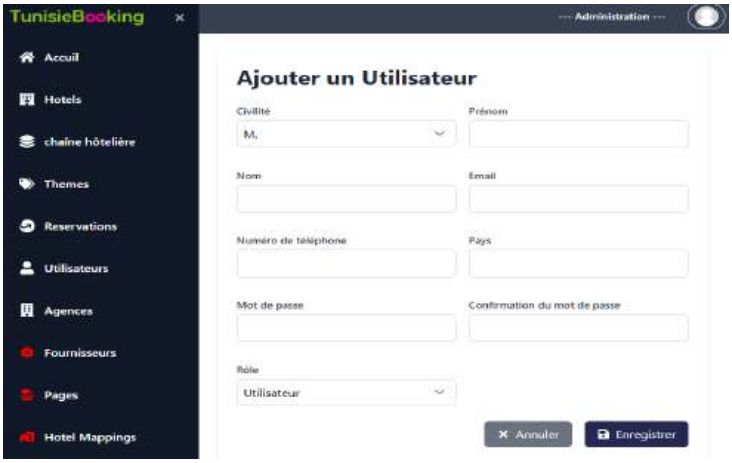


Fig. 3.10. Add New User Interface

Conclusion

This first sprint established the foundation of the platform by implementing a secure and extensible authentication system. Key features such as user registration, login, email verification, password reset, and user role management were developed. The use of diagrams supported iterative refinement of the design and helped validate the core user flows.

Chapter 4

Study and implementation of sprint 2

Contents

Introduction **38**

4.1 Sprint 2 Objective **38**

4.2 Sprint 2 Backlog **39**

4.3 Design **39**

4.3.1 Use Case Diagram 40

4.3.2 Class diagram 43

4.3.3 Sequence diagram 44

4.4 Implementation **45**

Conclusion **50**

Introduction

This chapter presents Sprint 2, focusing on the core features of hotel availability, search, and booking. It includes the backlog, use case diagram, and functional design components developed during this sprint.

4.1 Sprint 2 Objective

Sprint 2 is dedicated to the implementation of hotel-related functionalities, including hotel availability APIs, hotel search features, and the booking system.

4.2 Sprint 2 Backlog

The table below outlines the tasks planned for Sprint 2.

User Story	**Tasks**	**Estimate (Days)**
As a super admin or admin, I want to search, compare, and book hotels across multiple suppliers so that I can assist clients and confirm their reservations.	• Integrate supplier APIs • Display and compare hotel data • Validate room availability structure • Super admin booking logic • Reservation confirmation process • Test all flows • Test API responses	7
As a client or visitor, I want to search for hotels using filters so that I can find accommodations that match my needs.	• Search UI (form + filters) • Backend filtering logic • Pagination + results view • Input validation • Testing	7
As a client or visitor, I want to book hotel rooms so that I can confirm and reserve my desired stay.	• Booking form + reservation logic • Price and availability check • Invoice generation • Supplier selection (admin only) • Tests	8
As an admin or super admin, I want to manage reservations and confirm invoices so that I can oversee booking operations and client billing.	• Reservation list UI • Reservation status update • Invoice validation • Admin confirmation logic • Testing	6

Tab. 4.1. Sprint 2 Backlog

4.3 Design

In order to model our system, we will successively present the use case diagram, the class diagram, and the sequence diagram.

4.3.1 Use Case Diagram

The following use case diagram illustrates the main actions performed during hotel search and booking, as implemented in Sprint 2.

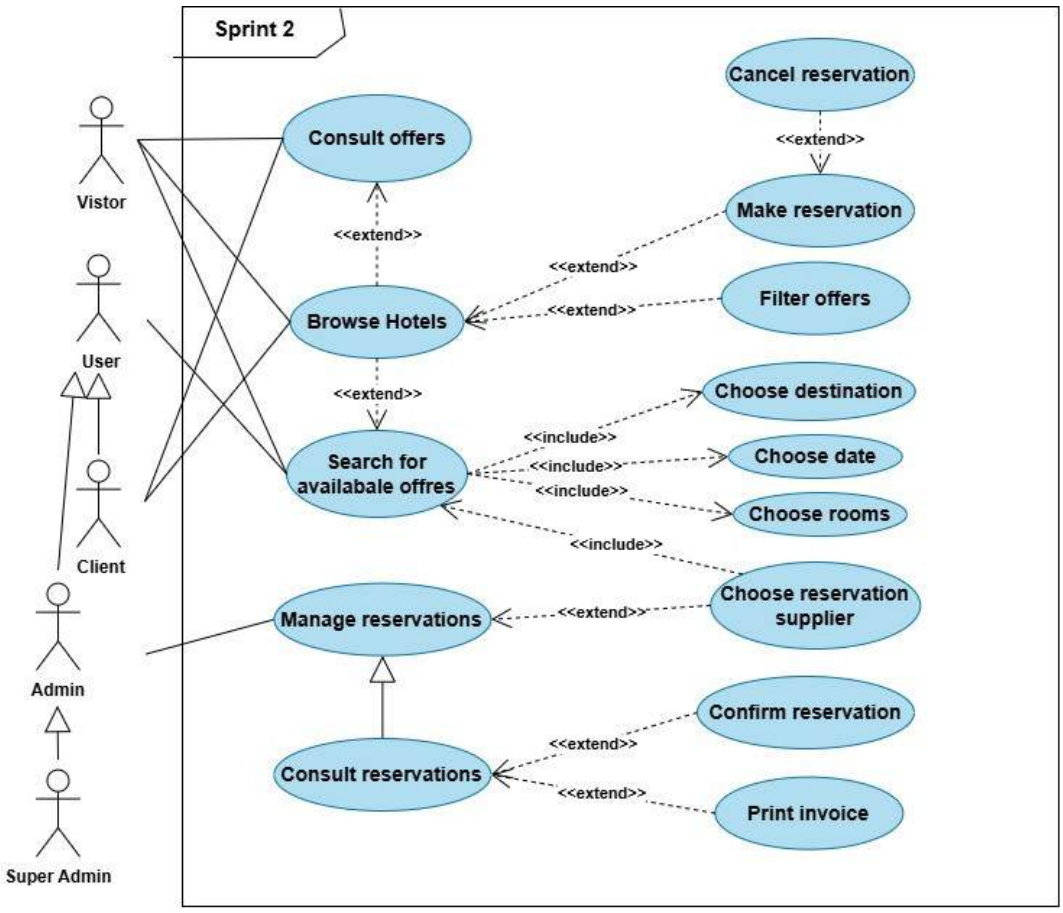


Fig. 4.1. Use Case Diagram – Hotel Search and Booking (Sprint 2)

Textual Description of the Use Case "Consult Offers"

Title	Consult Offers
Actor	Visitor / Client
Pre-condition	Promotional hotel offers exist on the platform.
Post-condition	User views the hotels linked to an offer and may proceed to "Browse Hotels".

Main Scenario	1. System displays the list of current offers. 2. User selects an offer. 3. System shows all hotels included in that offer. 4. User reviews each hotel's brief details. 5. User chooses a hotel and is redirected to "Browse Hotels".
Exception Scenarios	A1. **Offer not found:** System shows "Offer unavailable". A2. **Network error:** System asks the user to retry later.

Tab. 4.2. Textual description of the use case "Consult Offers".

Textual Description of the Use Case "Browse Hotels"

Title	Browse Hotels
Actor	Visitor / Client / Admin / Super Admin
Pre-condition	A list of hotels is available from a search or an offer.
Post-condition	User views detailed hotel information and may initiate a booking.
Main Scenario	1. System displays a list of hotels. 2. User selects a hotel. 3. System shows hotel details (photos, rooms, amenities, reviews). 4. User chooses a room and proceeds to "MakeReservation".
Exception Scenarios	B1. **Hotel no longer available:** System notifies the user and updates the list.

Tab. 4.3. Textual description of the use case "Browse Hotels".

Textual Description of the Use Case "Search for Available Offers"

Title	Search for Available Offers
Actor	Visitor / Client / Admin / Super Admin
Pre-condition	User is on the platform's search page.
Post-condition	A list of matching hotels is displayed; user may continue to "Browse Hotels".

Main Scenario	1. User enters destination, dates, and number of guests. 2. User submits the search. 3. System queries suppliers and returns available hotels. 4. If Admin / Super Admin, user can compare suppliers and pick one. 5. User applies sort/filter options. 6. User selects a hotel → "Browse Hotels".
Exception Scenarios	C1. **No results:** System shows "No offers available". C2. **Invalid input:** System asks for valid destination/dates.

Tab. 4.4. Textual description of the use case "Search for Available Offers".

Textual Description of the Use Case "Manage Reservations"

Title	Manage Reservations
Actor	Admin / Super Admin
Pre-condition	At least one reservation exists in the system.
Post-condition	Reservation data is modified (status, details) and/or an invoice is printed.
Main Scenario	1. Admin opens the "Manage Reservations" page. 2. Admin searches or filters reservations (reference, status, client). 3. System lists matching reservations. 4. Admin selects a reservation to view details. 5. Admin updates status (confirm, cancel) or edits details. 6. Admin prints the invoice if required. 7. System saves changes.
Exception Scenarios	D1. **Reservation not found:** System shows "No matching reservation". D2. **Invalid update:** System blocks the action and displays an error. **Print error:** System cannot generate the invoice PDF.

Tab. 4.5. Textual description of the use case "Manage Reservations".

4.3.2 Class diagram

This class diagram represents the architecture of a hotel reservation management system with several actors and entities. Compared with Sprint 1, new classes have been added, notably Reservation, which groups together information relating to hotel reservations, linked to Hotels, themselves located in a City, associated with a Theme, and which may belong to a Hotel_chain. The Agency and Supplier classes are used to manage offers, each maintaining several relationships with hotels and reservations.

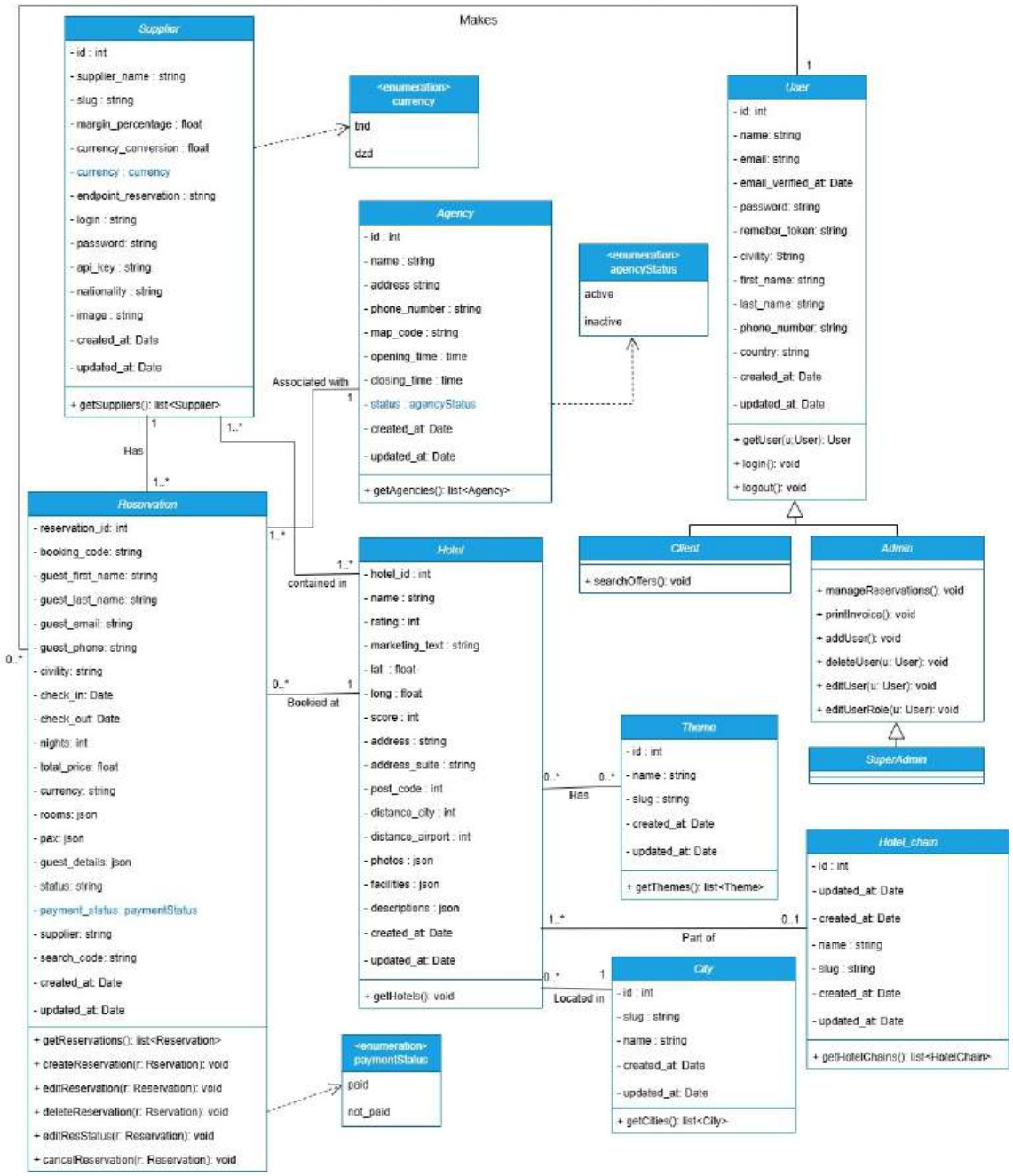


Fig. 4.2. Class diagram – Hotel Search and Booking (Sprint 2)

4.3.3 Sequence diagram

Reservation flow

This diagram illustrates the booking workflow for a user (visitor or client) on the TunisieBooking platform. The process begins with the user searching for hotel offers by providing the destination, travel dates, and number of rooms. If available, the system displays the results, allowing the user to filter and select an offer. Upon selection, the session code from the search is sent to the TunisieBooking system to fetch hotel information via the *Devis API*. The user then fills in reservation details, which are sent through the *CreateReservation API*. The system processes the reservation, stores it in the database, generates an invoice, and sends a confirmation email. If no offers match the search criteria, an appropriate message is displayed, and the user must wait for administrative confirmation. The diagram also presents an optional scenario where the user may cancel a reservation, prompting the system to update the reservation status accordingly.

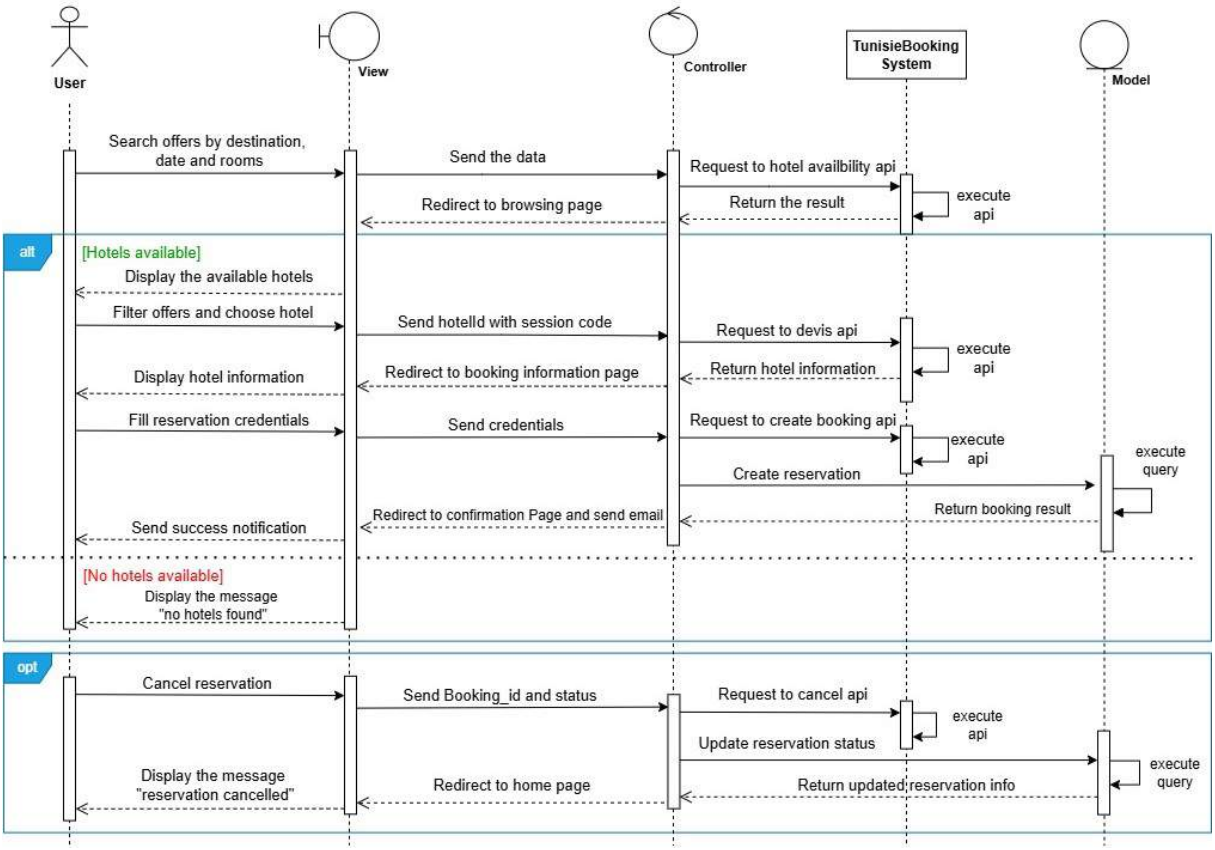


Fig. 4.3. Hotel Search and Booking Sequence diagram

Reservation flow for admin

This second diagram focuses on the reservation process carried out by an administrator (admin or super admin) on the TunisieBooking dashboard. The admin begins by choosing a reservation supplier. Afterward, the admin proceeds with the same booking workflow as a regular user, including offer selection and reservation creation. Additionally, the admin has the ability to change the status of a reservation. If the reservation is confirmed, the system sends a request to the *Destockage API* to validate the booking and update the reservation status in the database. In case of cancellation, a cancellation request is sent, and the reservation status is updated accordingly. A confirmation message is then displayed to the admin.

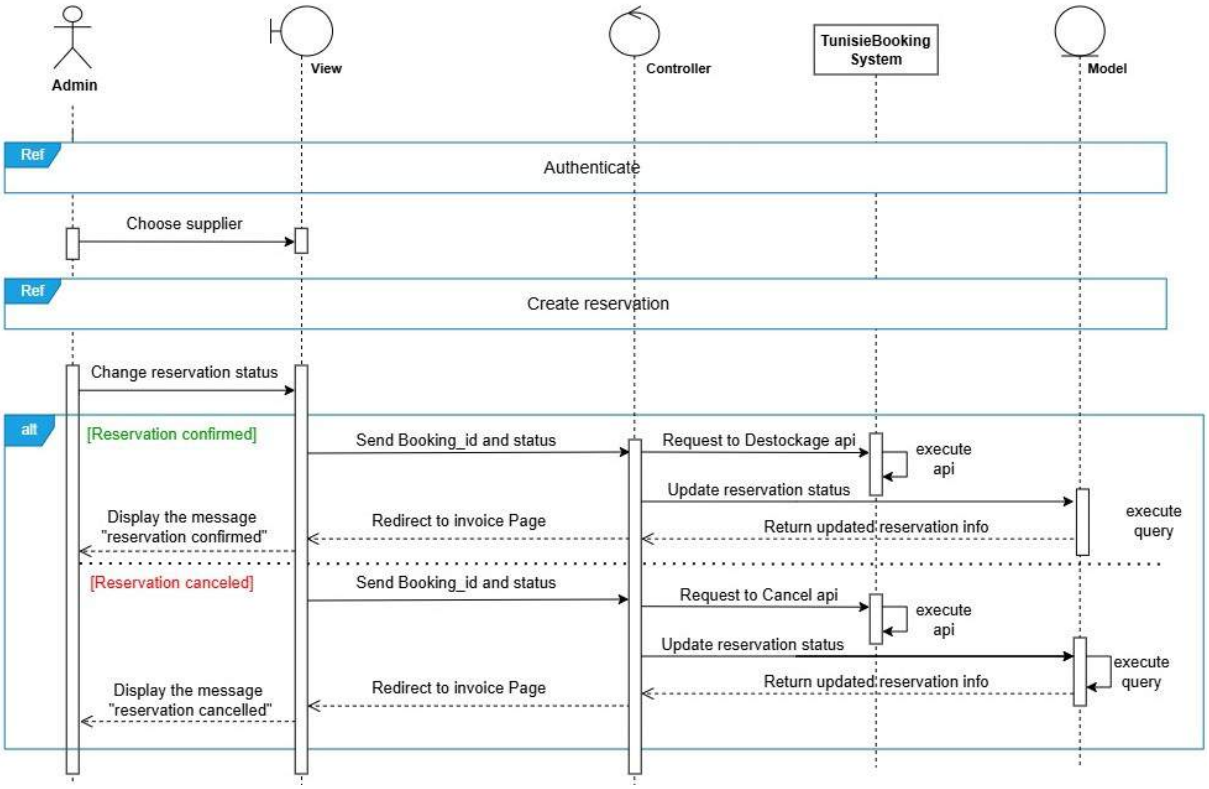


Fig. 4.4. Hotel Search and Booking Sequence diagram for admin

4.4 Implementation

This section presents the various user interfaces developed during Sprint 2. These interfaces focus on hotel search, availability browsing, booking, and administrative functionalities. Together, they enable a complete hotel reservation flow for both end users and administrators.

Search Interface: Users can search for hotels by selecting the destination, travel dates, and number of rooms.

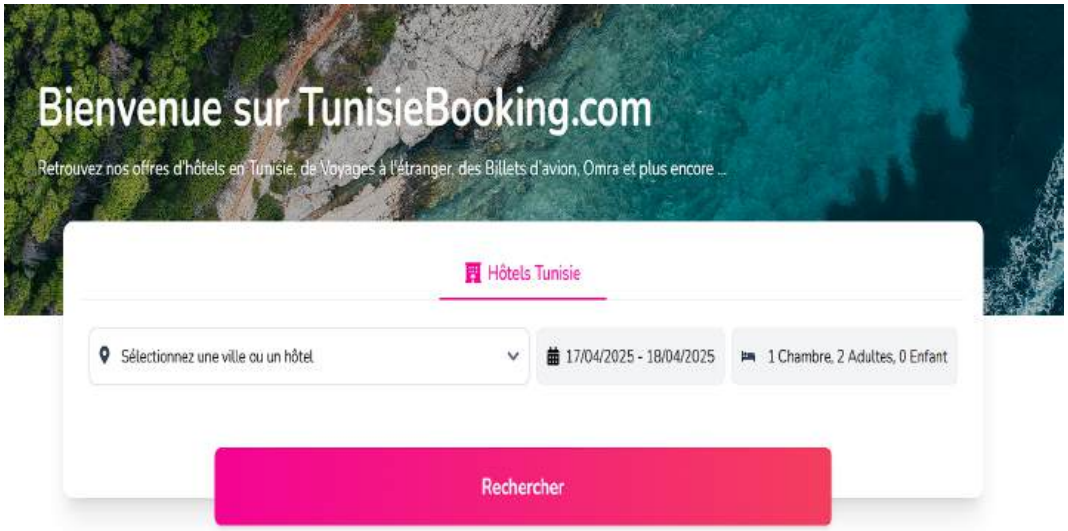


Fig. 4.5. Search Interface

Hotel Browsing Interface (Admin View): Admins can view hotel results with the additional ability to choose a reservation supplier for price comparison and booking flexibility.

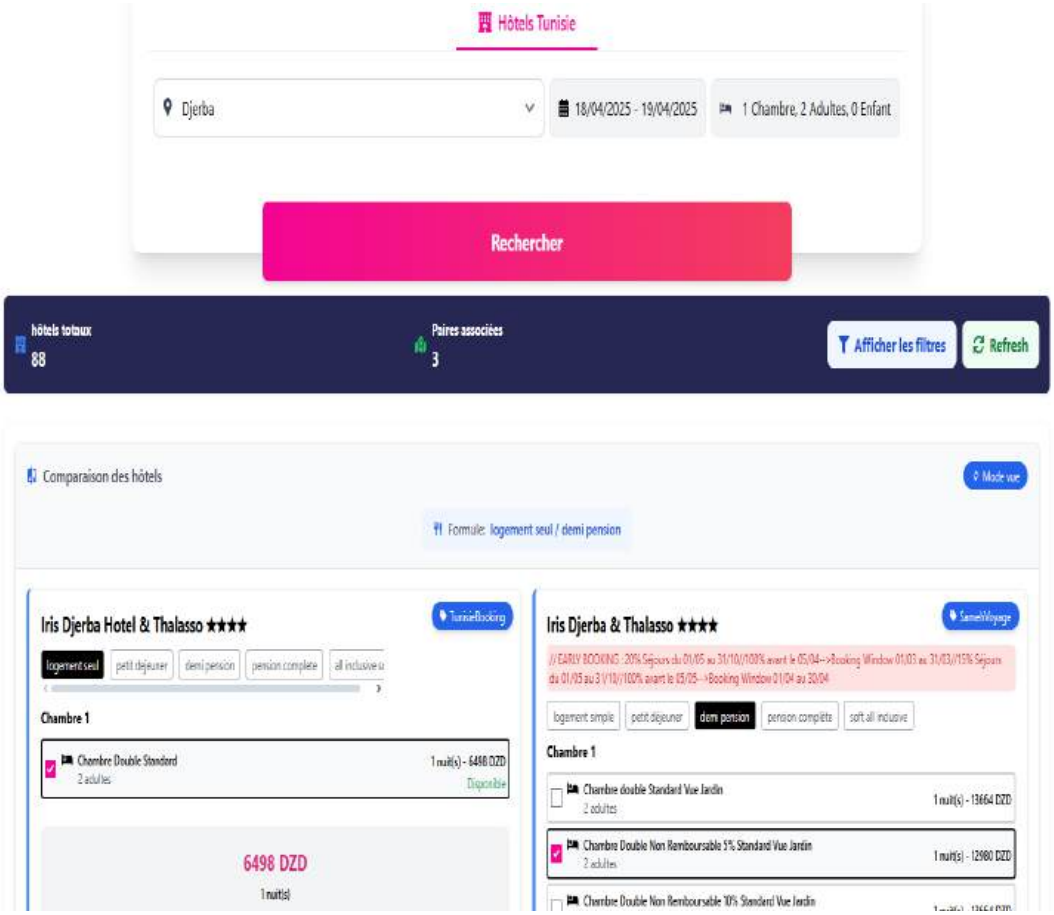


Fig. 4.6. Hotel Browsing Interface (Admin View)

Hotel Details Interface: Provides detailed information about a selected hotel, aiding users in the decision-making process.

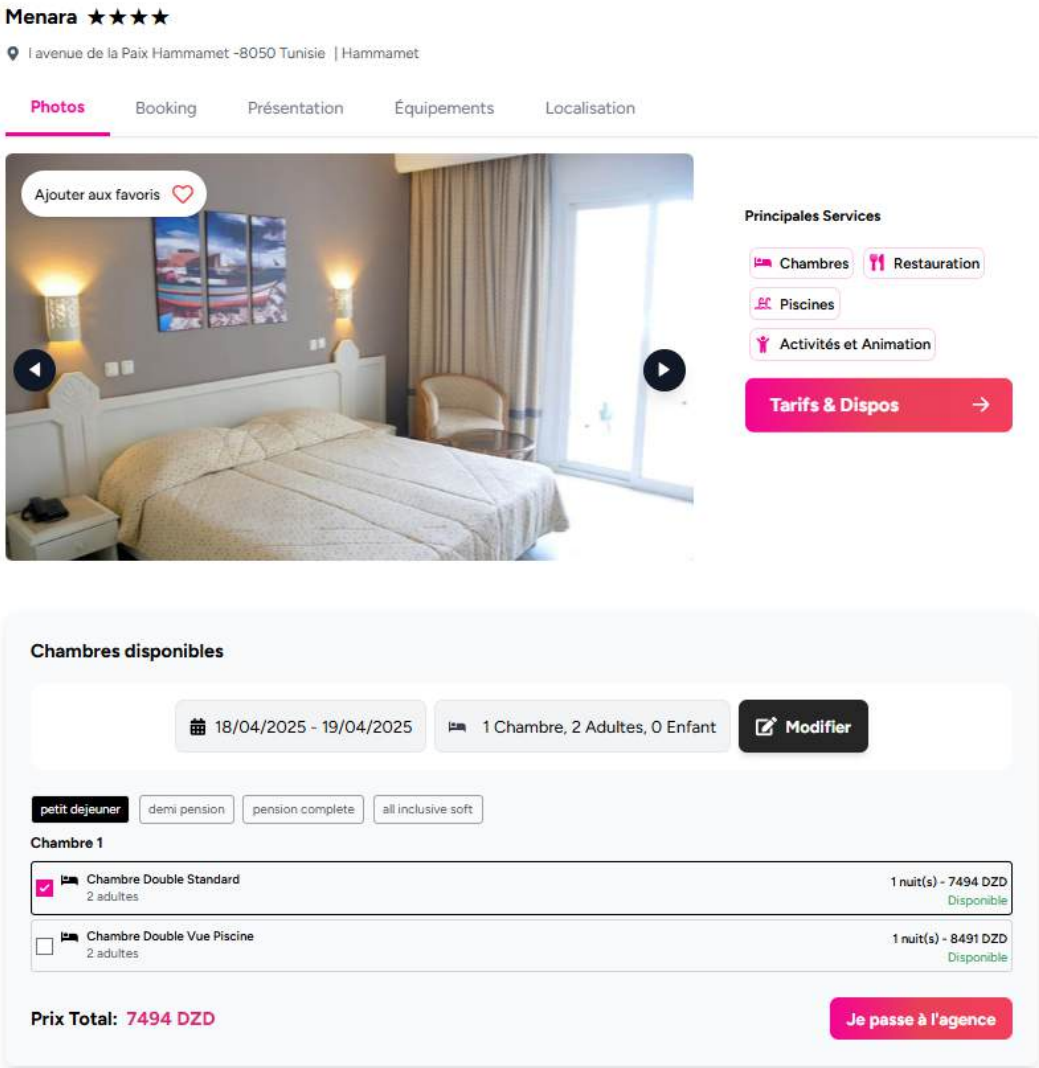


Fig. 4.7. Hotel Details Interface

Hotel Browsing Interface (Client View): After performing a search, users see a list of available hotels with filters to narrow results based on criteria like price, pension type, and room availability.

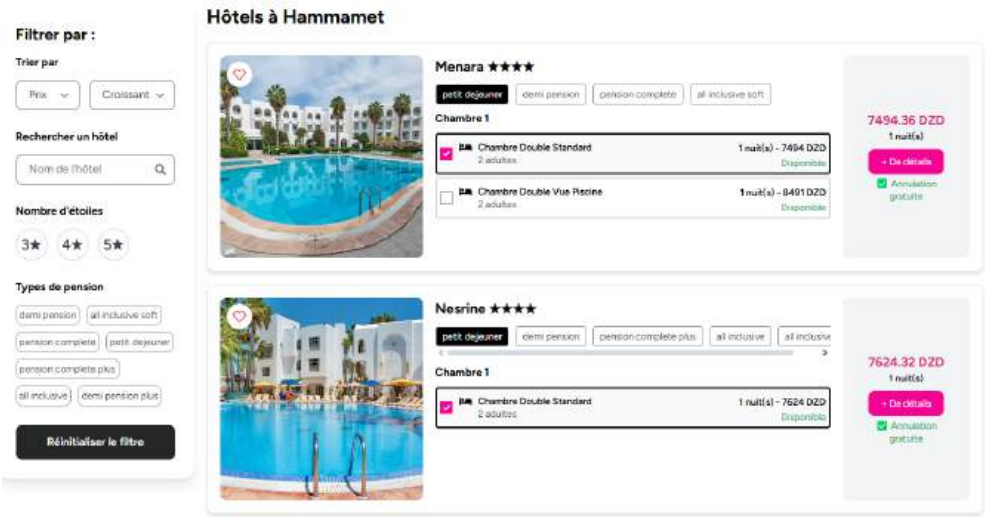


Fig. 4.8. Hotel Browsing Interface (Client View)

Reservation Interface: Users finalize their booking by entering personal and travel details. Once submitted, a confirmation email is sent automatically.

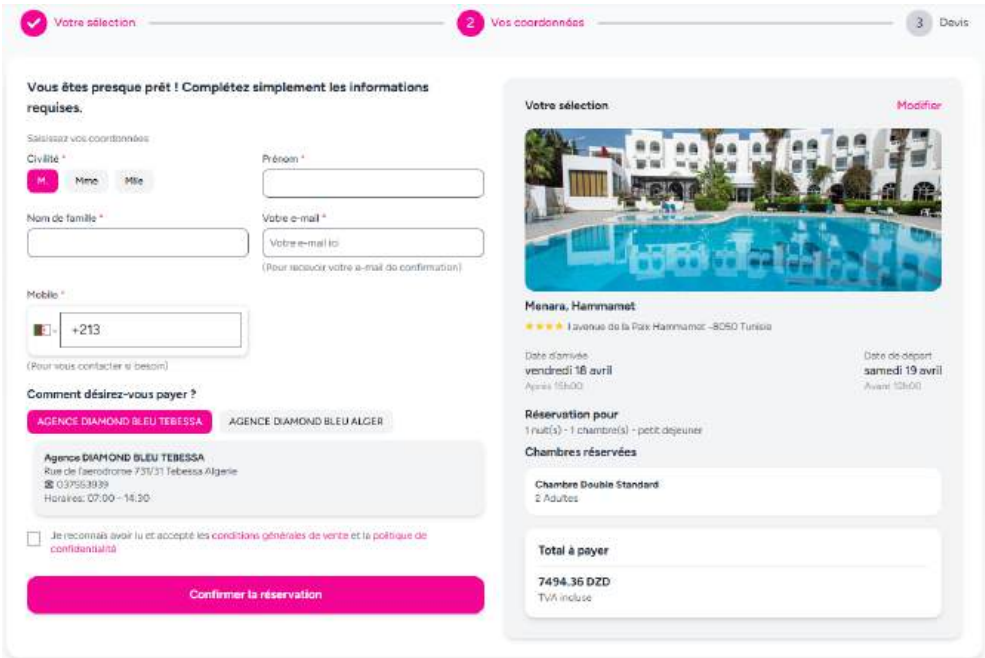


Fig. 4.9. Reservation Interface

Invoice and Reservation Status Interface: Administrators can generate and print detailed invoices and manage reservation statuses. If a client fails to arrive at the agency at least one hour before the scheduled time, the system redirects the admin to the hotel availability page when attempting to confirm the booking, prompting a new search.

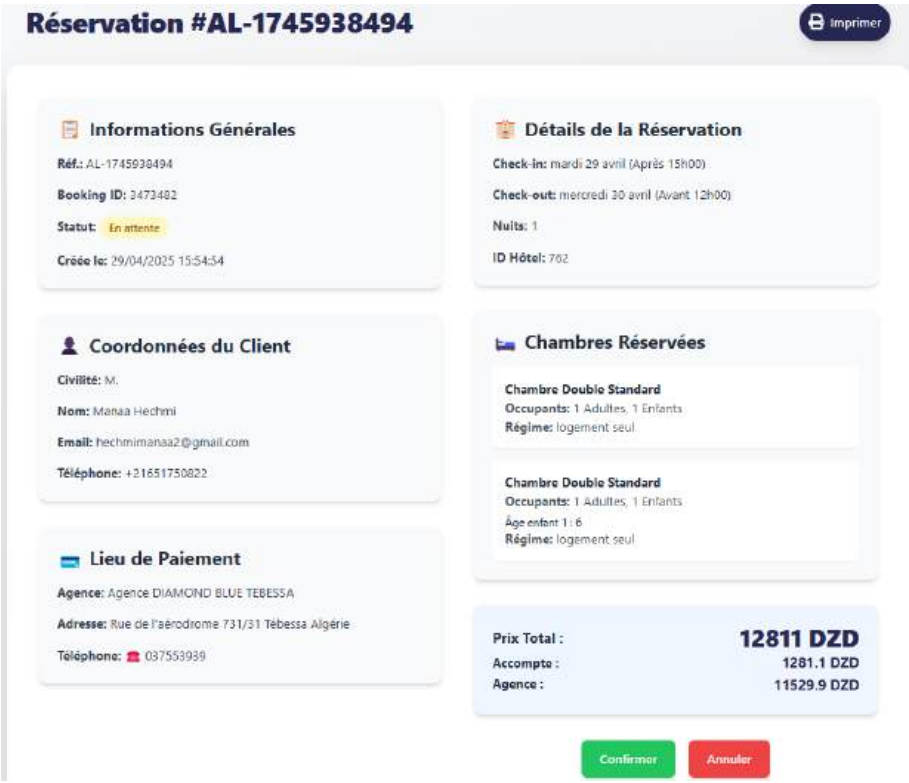


Fig. 4.10. Invoice and Reservation Status Interface

Confirmation Email: Summarizes booking details such as travel dates, hotel info, and total cost.

Fig. 4.11. Confirmation Email

Conclusion

In Sprint 2, we began by clearly defining our objectives and establishing the product backlog related to hotel booking functionalities. We then proceeded to design the system using UML diagrams, including use case, class, and sequence diagrams. Finally, we implemented key modules such as hotel availability through external APIs, dynamic search capabilities, the full reservation process, and invoice generation.

Chapter 5

Study and Implementation of Sprint 3

Contents

Introduction **51**

5.1 Sprint 3 Objective **51**

5.2 Sprint 3 Backlog **52**

5.3 Design **52**

5.3.1 Use Case Diagram 52

5.3.2 Class diagram 56

5.3.3 Sequence diagram 57

5.4 Implementation **58**

Conclusion **62**

Introduction

This chapter presents Sprint 3, focusing on administrative tools including dashboards, supplier management, and hotel mapping. It includes the sprint backlog, the use case diagram, and the design work completed during this phase.

5.1 Sprint 3 Objective

The sprint 3 focuses on the establishment of the admin dashboard environment, including the development of statistics and data listing pages, as well as the implementation of hotel mapping logic to unify data across suppliers.

5.2 Sprint 3 Backlog

The table below outlines the tasks planned for Sprint 3.

User Story	**Tasks**	**Estimate (Days)**
As an admin or super admin, I want to view statistics so that I can monitor bookings and system usage.	• Design dashboard UI • Display booking and user stats • Integrate chart components • Role-based access • Testing	8
As an admin or super admin, I want to manage agencies and hotel details so that I can keep information accurate and updated.	• Agency CRUD pages • Hotel edit form (description, details) • View hotel chains and themes • Search/filter logic • Testing	8
As a super admin, I want to manage suppliers so that I can control API access, margins, and conversion settings.	• Supplier CRUD UI • Add API endpoints • Set conversion and margin values • Toggle supplier status • Tests	7
As a super admin, I want to map hotels from different suppliers so that I can unify duplicate listings.	• Fetch supplier hotel lists • Identify duplicates • Manual hotel linking UI • Save mapping data • Testing	7

Tab. 5.1. Sprint 3 Backlog

5.3 Design

The following section is dedicated to the design of UML diagrams, namely: use cases, class diagrams, and sequence diagrams.

5.3.1 Use Case Diagram

The following diagram illustrates the core administrative functions developed during Sprint 3, including supplier management and hotel mapping.

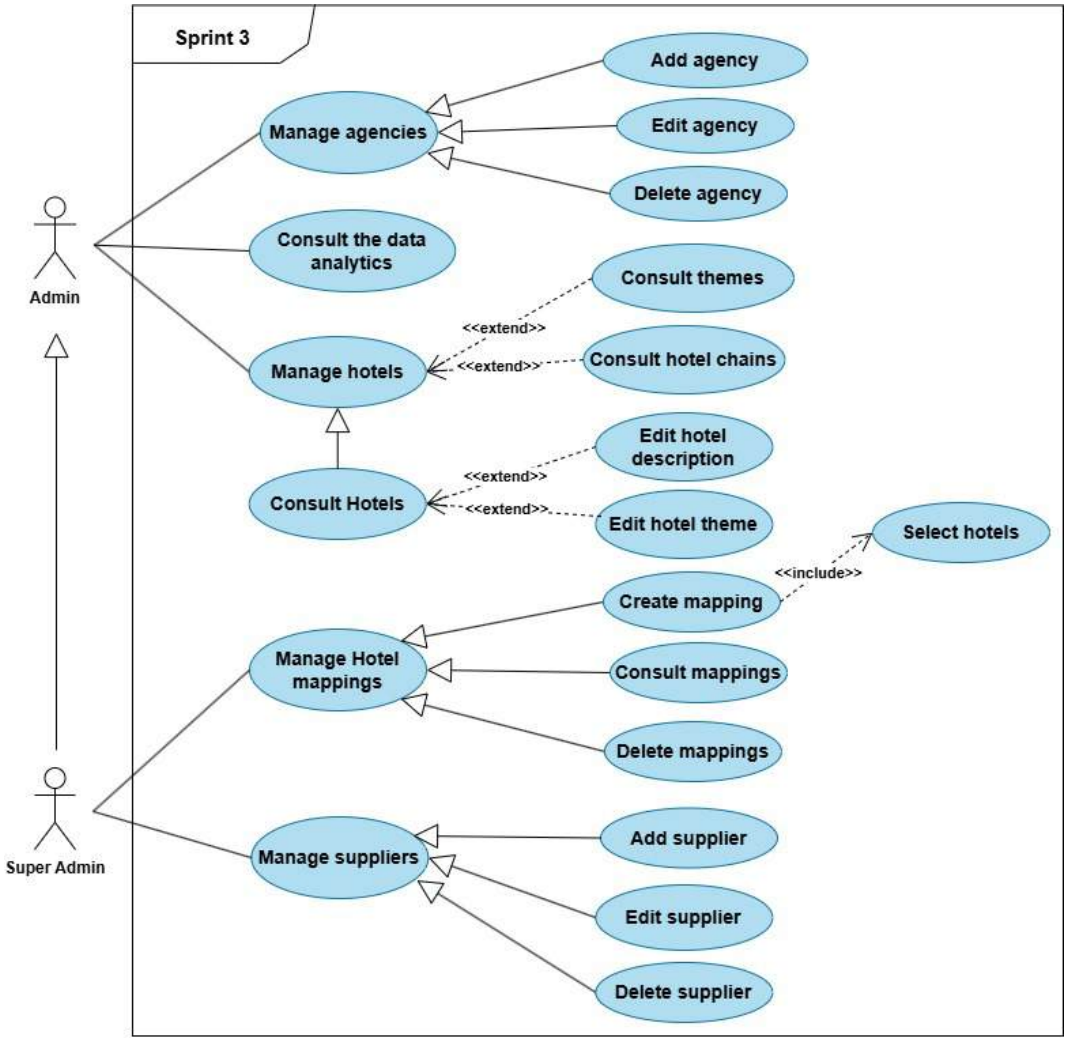


Fig. 5.1. Use Case Diagram – admin Environment (Sprint 3)

Textual Description of the Use Case "Manage Agencies"

Title	Manage Agencies
Actor	Admin
Pre-condition	Admin is authenticated.
Post-condition	Agencies are created, updated, or deleted in the system.

Main Scenario	1. Admin accesses the agency management interface. 2. Admin chooses to add, edit, or delete an agency. 3. Admin enters or modifies agency information. 4. System validates the data. 5. System updates the agency list accordingly and confirms the operation.
Exception Scenarios	A1. **Invalid input:** System displays an error such as ”Invalid agency details”. A2. **Database error:** System shows a message like ”Failed to update database”.

Tab. 5.2. Textual description of the use case “Manage Agencies”.

Textual Description of the Use Case “Manage Hotels”

Title	Manage Hotels
Actor	Admin
Pre-condition	Admin is authenticated and has access to hotel data.
Post-condition	Hotel data (description, theme, chain) is modified in the system.
Main Scenario	1. Admin opens the hotel management interface. 2. Admin selects a hotel to view or edit. 3. Admin modifies the hotel description, theme, or chain. 4. System validates the changes. 5. System saves the modifications and confirms success.
Exception Scenarios	B1. **Invalid input:** System displays an error like ”Hotel description required”. B2. **Database error:** System shows an error message such as ”Unable to save changes”.

Tab. 5.3. Textual description of the use case “Manage Hotels”.

Textual Description of the Use Case "Manage Hotel Mappings"

Title	Manage Hotel Mappings
Actor	Super Admin
Pre-condition	Super Admin is authenticated and hotel supplier data is available.
Post-condition	Mappings between hotels and suppliers are created, consulted, or deleted.
Main Scenario	1. Super Admin accesses the hotel mappings interface. 2. Super Admin chooses to create, consult, or delete a mapping. 3. For creation, Super Admin selects hotels and defines the mapping. 4. System validates the data. 5. System saves or removes the mapping and confirms the operation.
Exception Scenarios	C1. **Invalid input:** System shows "Invalid mapping details". C2. **Database error:** System displays "Failed to update mapping".

Tab. 5.4. Textual description of the use case "Manage Hotel Mappings".

Textual Description of the Use Case "Manage Suppliers"

Title	Manage Suppliers
Actor	Super Admin
Pre-condition	Super Admin is authenticated.
Post-condition	Suppliers are added, updated, or deleted in the system.
Main Scenario	1. Super Admin accesses the supplier management interface. 2. Super Admin selects to add, edit, or delete a supplier. 3. Super Admin provides or modifies supplier information. 4. System validates the input. 5. System updates the database and confirms the action.
Exception Scenarios	D1. **Invalid input:** System displays "Supplier name is required" or similar. D2. **Database error:** System displays "Failed to update database".

Tab. 5.5. Textual description of the use case "Manage Suppliers".

5.3.2 Class diagram

Use case diagrams provide an overview of a software system's functional behavior. The following figure illustrates the use case diagram for Sprint 3.

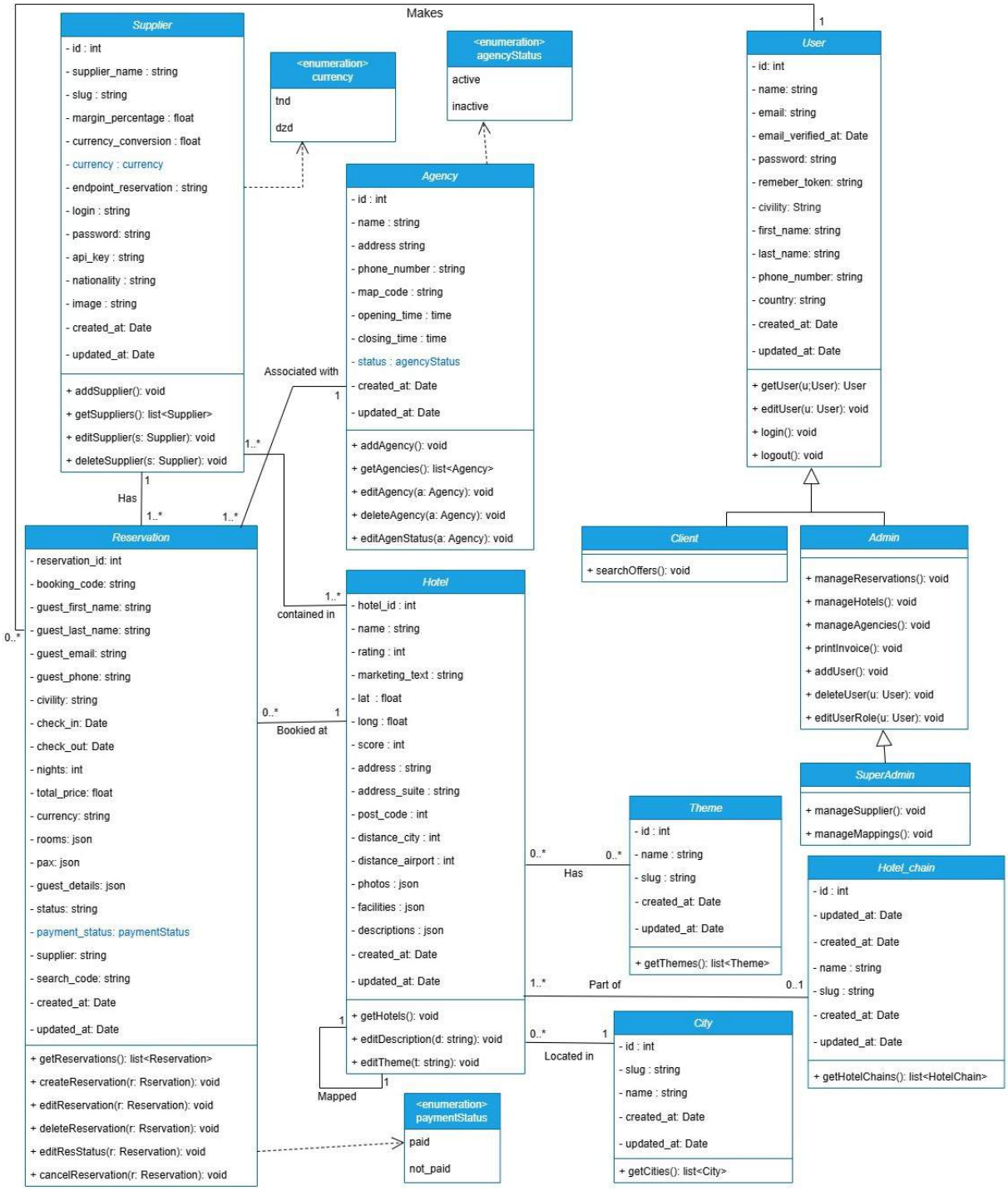


Fig. 5.2. Class Diagram – Administrators environment (Sprint 3)

5.3.3 Sequence diagram

This sequence diagram illustrates how an admin manages agencies through the system dashboard, including adding new agencies, updating existing details, and deleting agencies as needed.

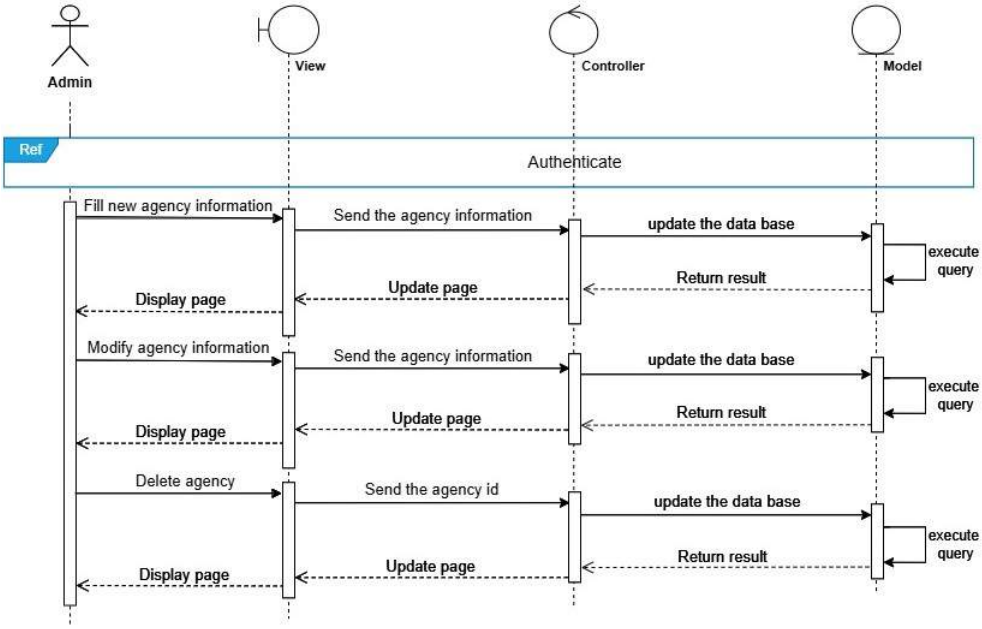


Fig. 5.3. Agency management Sequence diagram

This sequence diagram illustrates the supplier management process performed by a Super Admin, including adding, editing, or deleting suppliers, and managing hotel mappings.

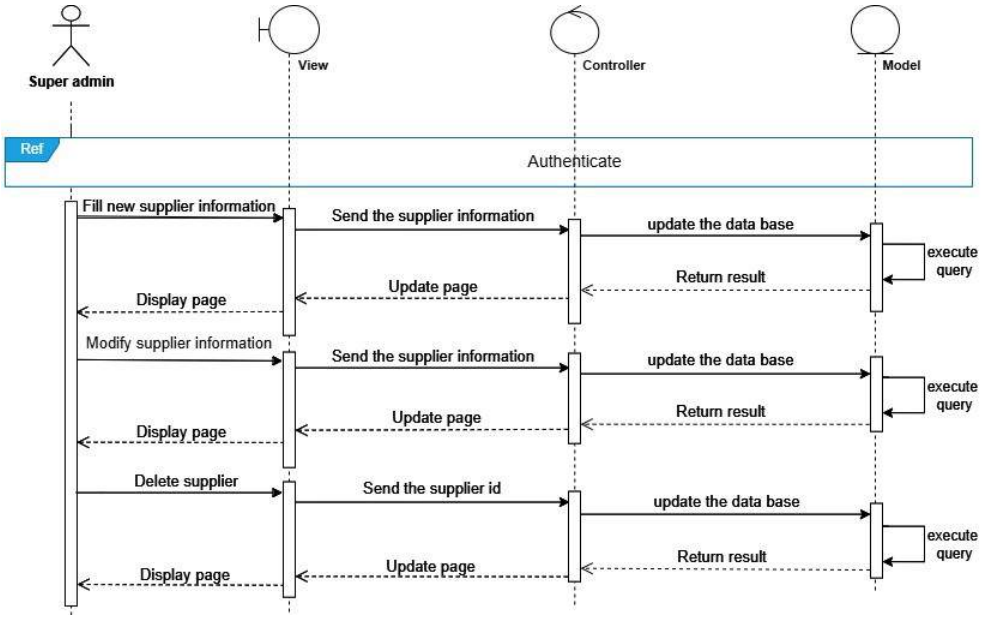


Fig. 5.4. Supplier management Sequence diagram

This sequence diagram illustrates the hotel mapping management process carried out by a Super Admin, who can either select hotels to create new mappings or delete existing ones.

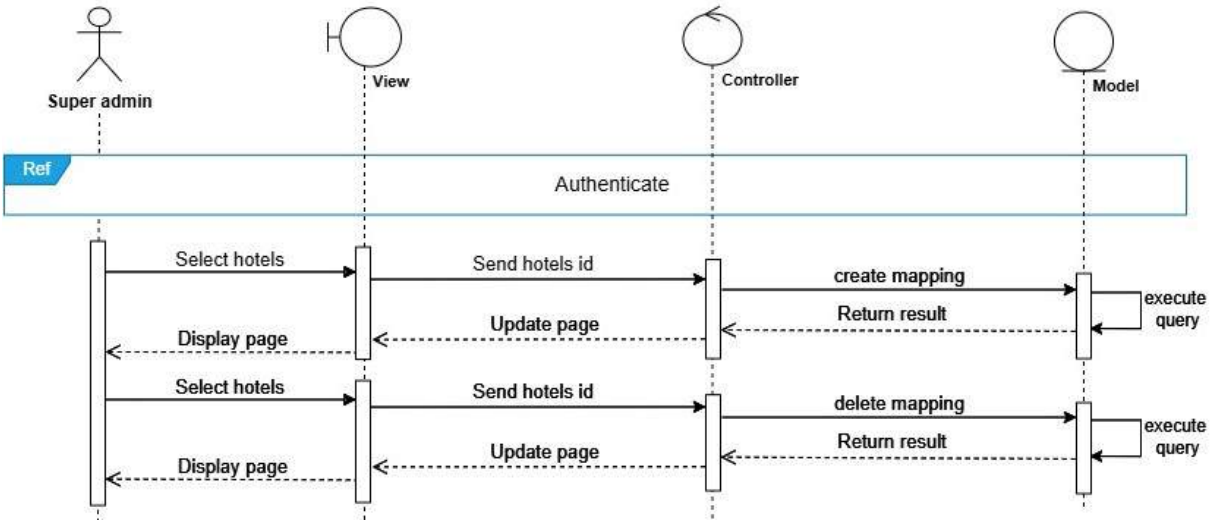


Fig. 5.5. Hotel mapping Sequence diagram

5.4 Implementation

Hotels Interface: Displays the total number of hotels and includes filtering options. Hotels are listed in a table format with action buttons for viewing or editing. An ”Apply” button confirms the selected filters.

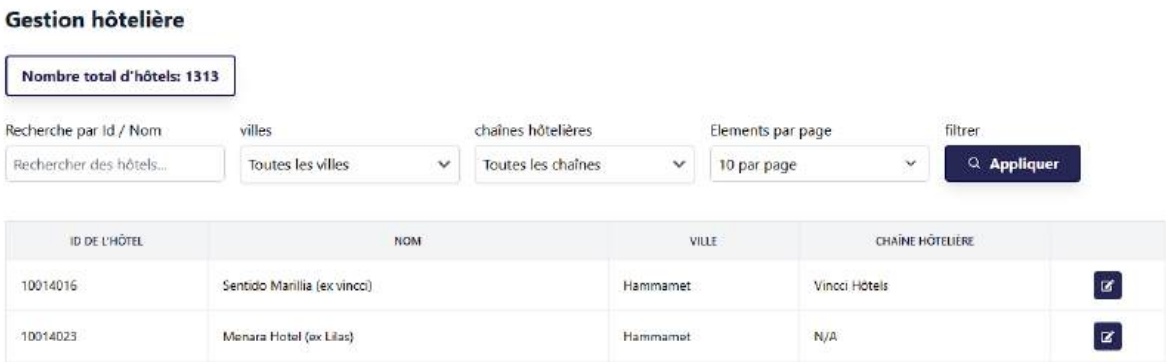


Fig. 5.6. Hotels interface

Administration Home Interface: Provides a concise overview of key metrics within the hotel management system. It displays essential data to help administrators monitor platform activity.

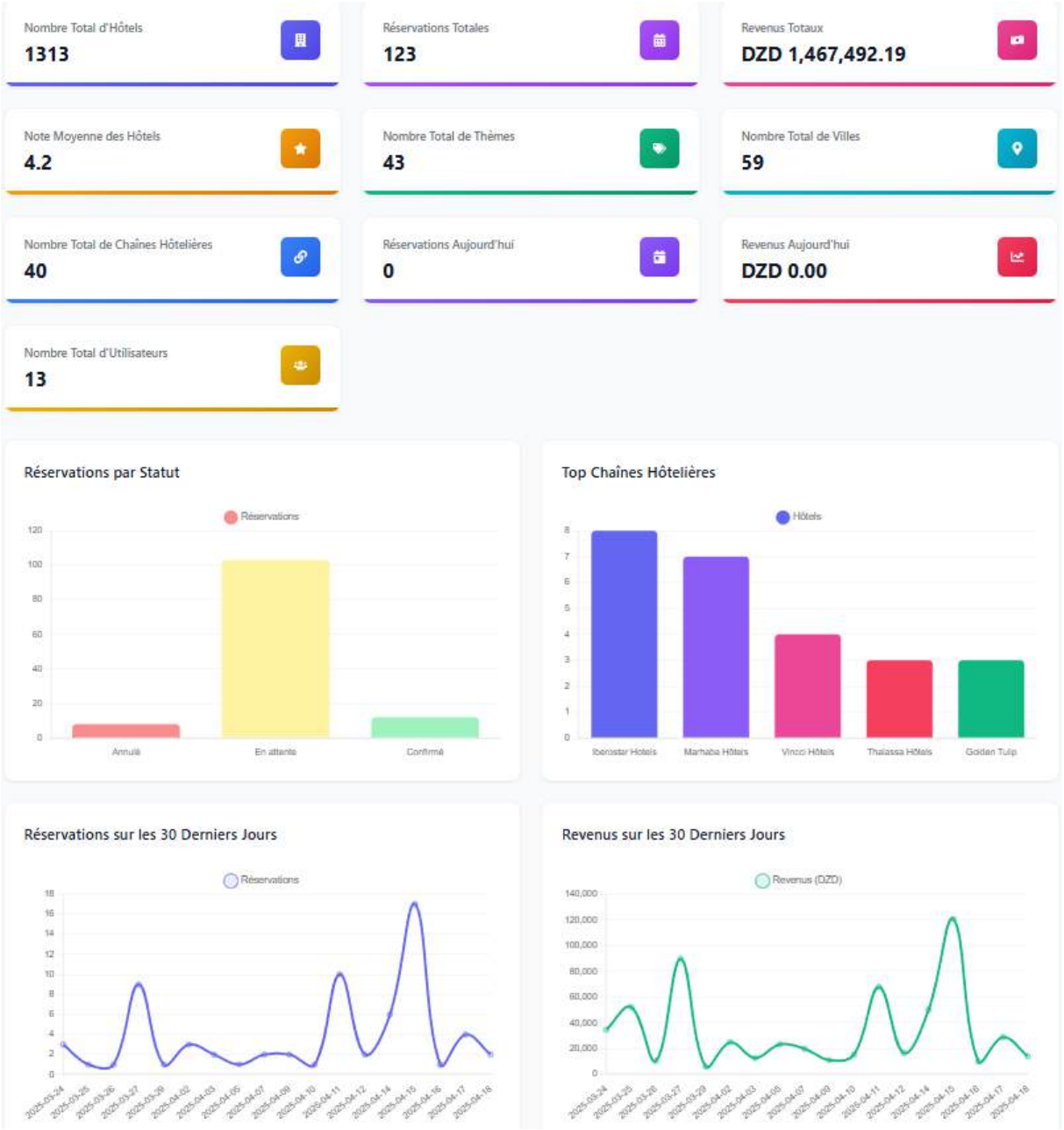


Fig. 5.7. Administration Home interface

Reservations Interface: Provides key statistics such as the number of total reservations, today's bookings, and total price. It supports booking additions via suppliers and includes a search feature for managing reservation records.

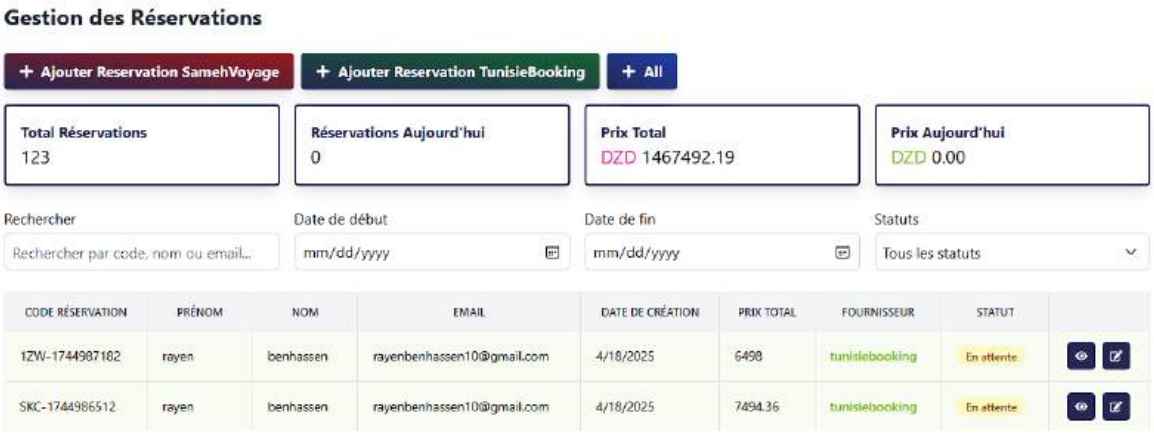


Fig. 5.8. Reservations interface

Hotel Chain Interface: Shows the number of hotel chains along with basic information related to each chain.

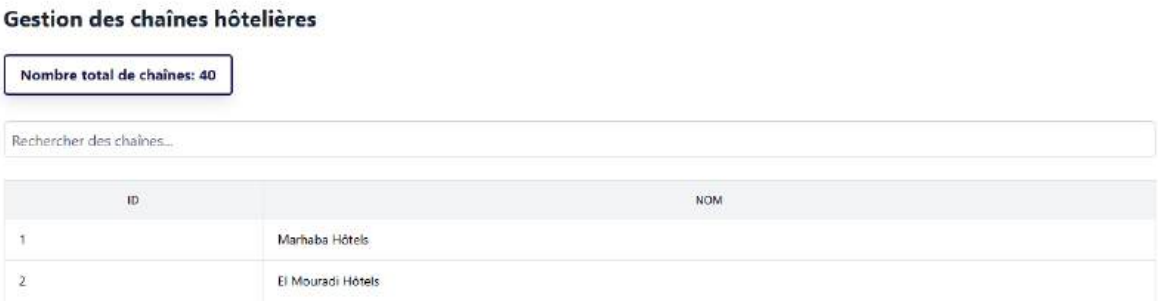


Fig. 5.9. Hotel Chain interface

Themes Interface: Displays the number of available themes and basic descriptive information about each one.

Fig. 5.10. Themes interface

Agency Interface: Displays the total number of agencies and includes a search bar for filtering. Agencies are shown in a table with action buttons for viewing or deleting, and a button to add new agency entries.

Fig. 5.11. Agency interface

Suppliers Interface: Allows the management of supplier records. Suppliers are listed on the left with options to delete or add entries. The right panel includes a form to edit details with a ”Save” button to confirm updates.

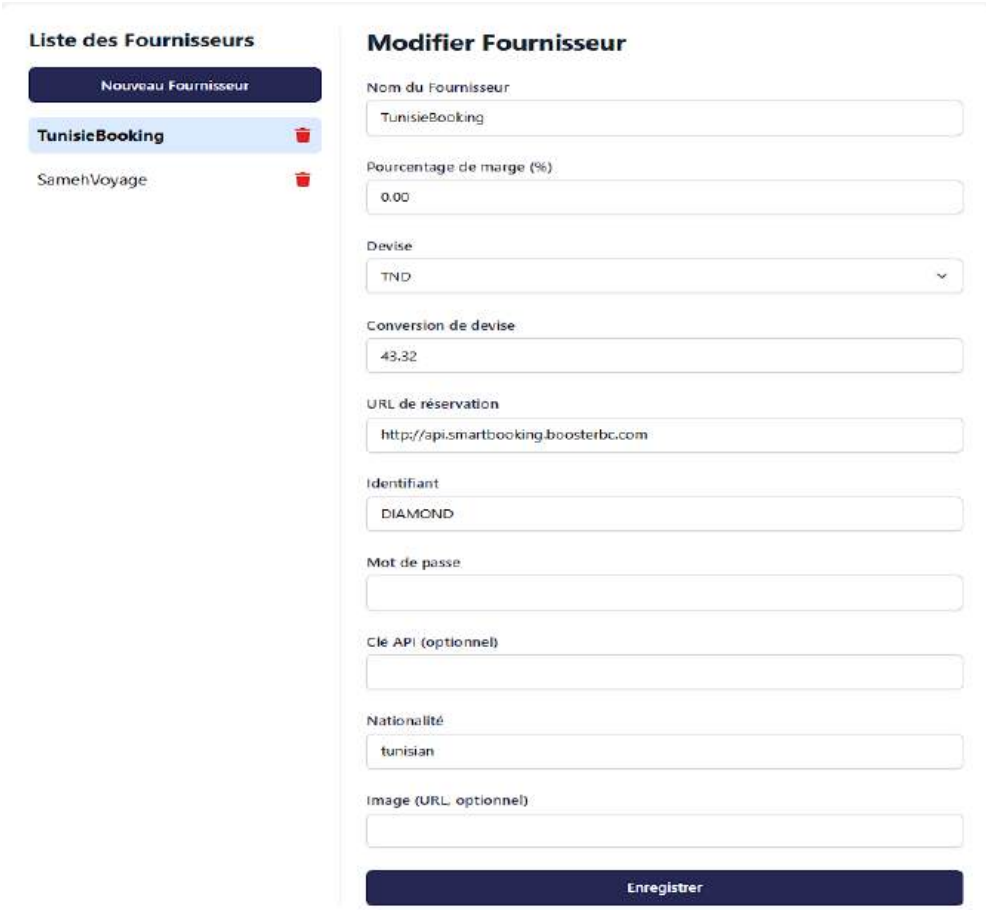


Fig. 5.12. Suppliers interface

Mapping Interface: Enables hotel mapping between suppliers. Search bars filter hotels by supplier, and mapping status is indicated. Admins can create or delete mappings, with a summary table and additional search functionality provided.

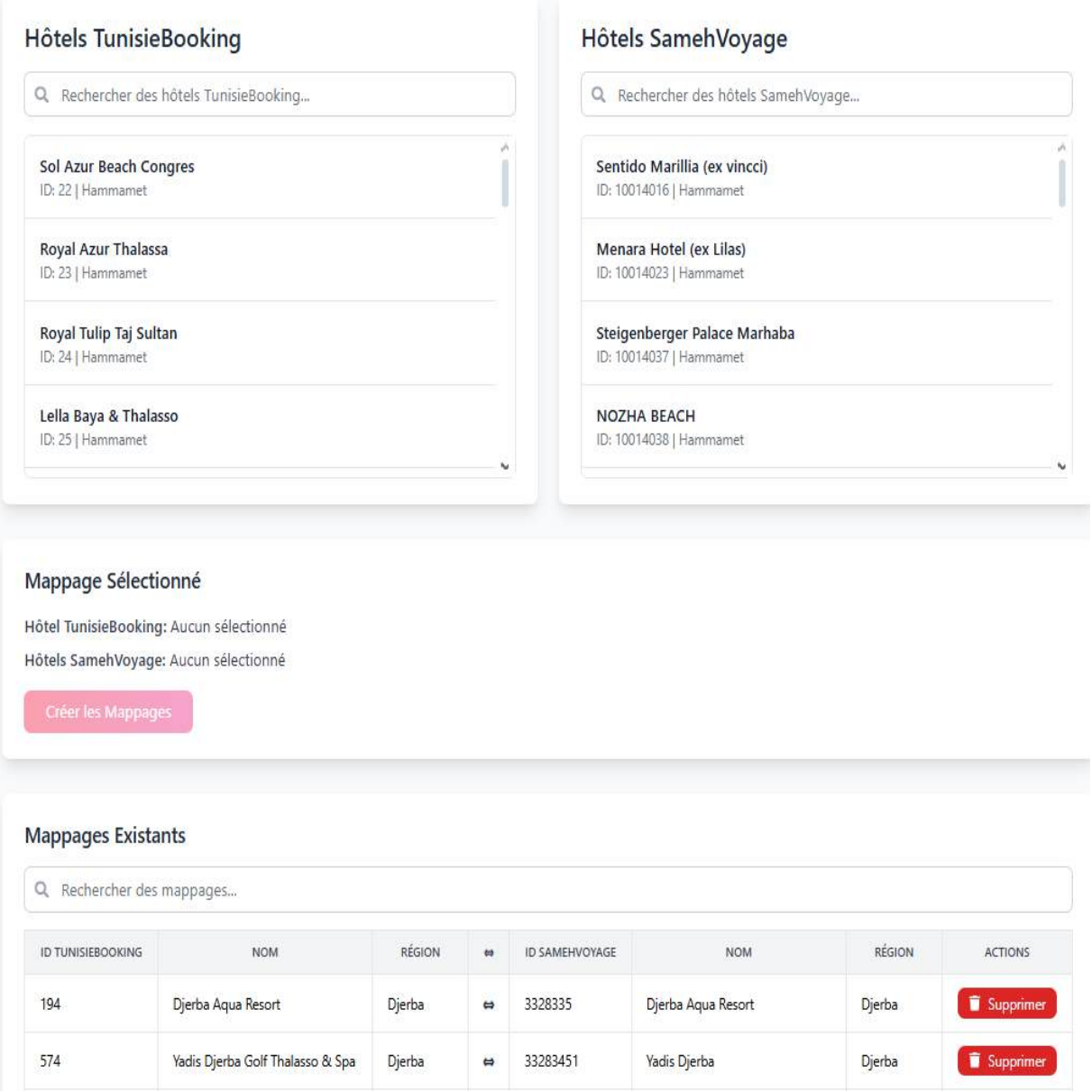


Fig. 5.13. Mapping interface

Conclusion

The third sprint introduced advanced administrative features, such as dashboards for both administrators and super administrators, supplier management, and hotel mapping logic. These tools provide improved control, visibility, and data consistency, supporting the efficient management of hotel data and enhancing the reliability of the booking system.

Chapter 6

Study and Implementation of Sprint 4

Contents

Introduction **63**

6.1 Sprint 4 Objective **63**

6.2 Sprint 4 Backlog **64**

6.3 Design **64**

6.3.1 Use Case Diagram 65

6.3.2 Class Diagram 67

6.3.3 Sequence Diagram 68

6.4 AI Integration **68**

6.5 Implementation **70**

Conclusion **73**

Introduction

This chapter presents Sprint 4, focusing on enhancing the client experience with personalized features and integrating an AI-powered chatbot for improved booking assistance.

6.1 Sprint 4 Objective

The objective of Sprint 4 is to enhance the client experience by providing a personalized dashboard, favorites management, and integrating an AI chatbot for real-time booking assistance.

6.2 Sprint 4 Backlog

The table below outlines the key tasks scheduled for Sprint 4, which aims to finalize the platform by empowering clients with personalized features and real-time assistance.

User Story	**Tasks**	**Estimate (Days)**
As a client, I want access to a personalized space so that I can view and manage my reservations and preferences.	• Design and implement client dashboard • Add profile editing features • Display reservation and search history • Enable reservation cancellation • Testing	12
As a client, I want to add hotels to a favorites list so that I can quickly access them later.	• Implement add/remove favorite logic • Create favorites list UI • Store and retrieve favorites per user • Testing	4
As a client, I want to interact with an AI chatbot so that I can get booking help and hotel suggestions.	• Integrate DeepSeek API • Build chatbot UI and chat history • Enable booking recommendation queries • Test chatbot responses • Final review	10

Tab. 6.1. Sprint 4 Backlog

6.3 Design

The following section is dedicated to the design of UML diagrams, namely: use cases, class diagrams, and sequence diagrams.

6.3.1 Use Case Diagram

The following diagram illustrates the interactions available to a client during Sprint 4, including access to the personal dashboard, managing favorites, canceling reservations, and using the AI-powered chatbot.

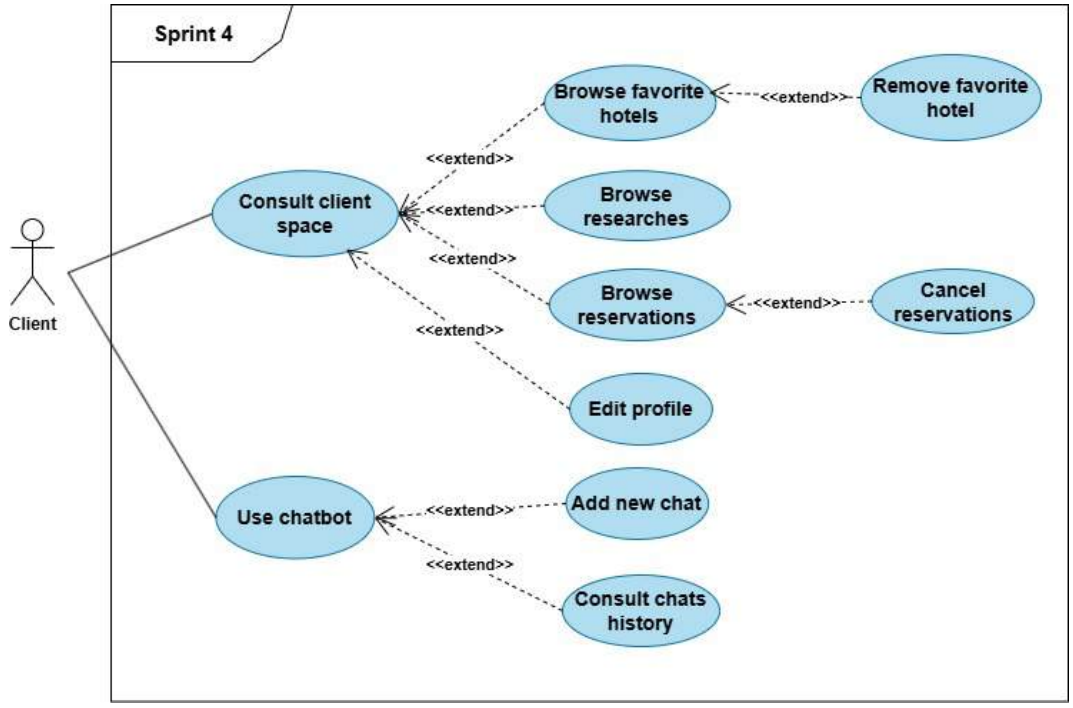


Fig. 6.1. Use Case Diagram – Client Space and AI Chatbot (Sprint 4)

Textual Description of the Use Case "Consult Client Space"

Title	Consult Client Space
Actor	Client
Pre-condition	Client is authenticated.
Post-condition	Client's profile, reservations, and favorites are viewed, updated, or deleted.

Main Scenario	1. Client accesses the client space interface. 2. Client chooses to view researches, reservations, or favorites. 3. Client may cancel a reservation. 4. Client may remove a favorite hotel. 5. Client may edit profile information. 6. System validates the input. 7. System updates data and confirms the action.
Exception Scenarios	C1. **Invalid input:** System displays "Invalid reservation ID". C2. **Database error:** System shows "Unable to process your request".

Tab. 6.2. Textual description of the use case "Consult Client Space".

Textual Description of the Use Case "Use Chatbot"

Title	Use Chatbot
Actor	Client
Pre-condition	Client is authenticated.
Post-condition	Client's chat messages are stored or retrieved from chat history.
Main Scenario	1. Client accesses the chatbot interface. 2. Client starts a new chat or consults previous chat history. 3. Client provides the necessary input. 4. System processes the request and updates or retrieves data. 5. System confirms the action (e.g., message sent).
Exception Scenarios	C1. **Invalid input:** System displays "Invalid chat input". C2. **Database error:** System displays "Unable to update or retrieve chat history".

Tab. 6.3. Textual description of the use case "Use Chatbot".

6.3.2 Class Diagram

The class diagram includes Chat, ChatMessage, and SearchHistory classes. A Client can start multiple Chats, each containing several ChatMessages from either the client or bot (via senderRole). Clients can also save favorite hotels through a link to the Hotel class.

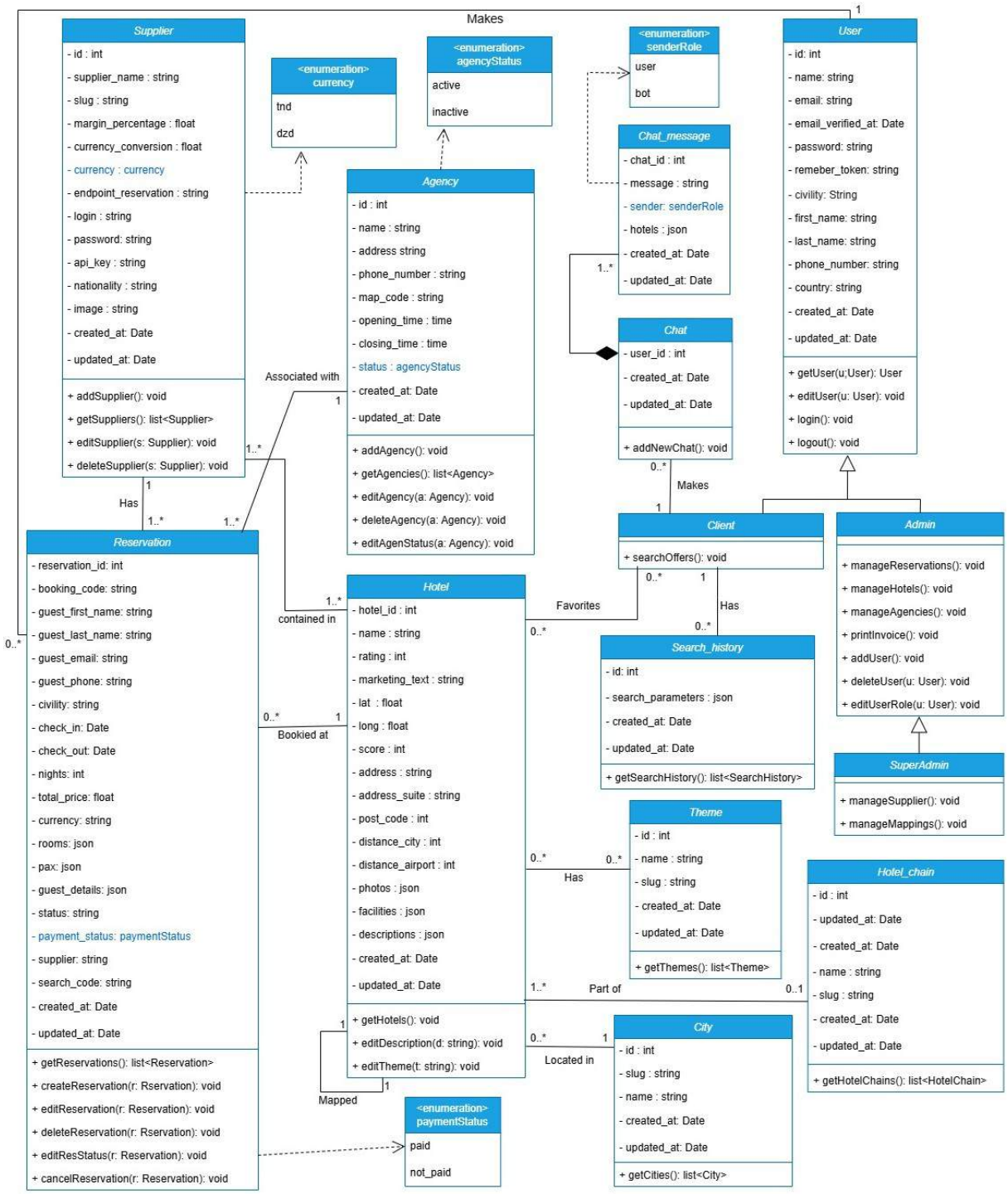


Fig. 6.2. Class Diagram – Client Space and Chatbot (Sprint 4)

6.3.3 Sequence Diagram

This sequence diagram illustrates the workflow of the chatbot system involving the User, View (interface), Chatbot Controller, DeepSeek Model, and Database. After authentication, the user enters a message, which is sent to the chatbot controller. The controller processes the message using the DeepSeek model, which may execute a prompt and request additional hotel data from the database if needed. The generated response is returned to the controller, which saves both the user's request and the chatbot's response to the database, and then sends the reply back to the view for display. Additionally, the user can consult their chat history by requesting it; the controller fetches the stored conversations from the database and displays them. The user also has the option to start a new chat session, triggering an action to create a new chat, with a confirmation sent back to the view.

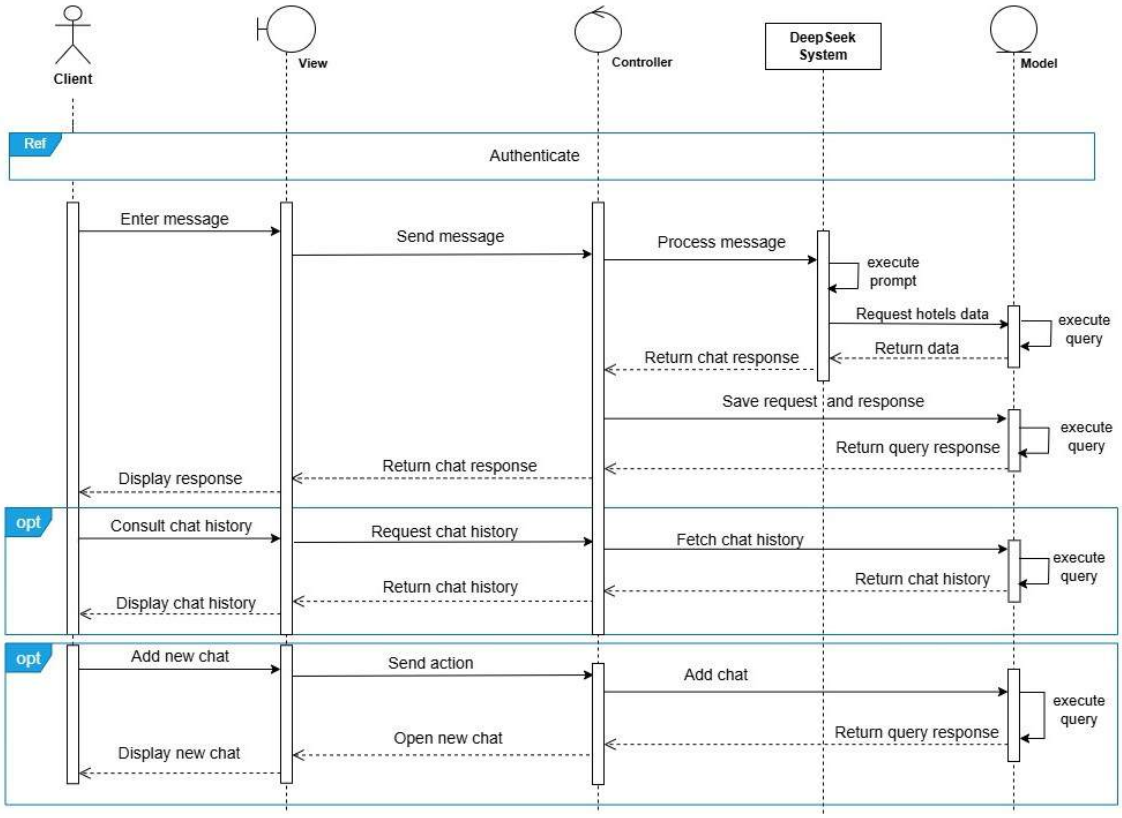


Fig. 6.3. Chatbot Sequence Diagram

6.4 AI Integration

Overview of DeepSeek

DeepSeek is an advanced large language model developed by the AI research company DeepSeek. Known for its strong performance in natural language understanding, multilingual

interaction, programming assistance, and reasoning tasks, DeepSeek is positioned as a competitive alternative to models like OpenAI's GPT-4 and xAI's Grok. It offers high performance at a lower cost, making it suitable for both individual developers and large-scale enterprise systems.

Integration Use Case

DeepSeek can be connected via API to deliver intelligent features such as conversational agents, content summarization, smart recommendations, and more. Its flexibility allows seamless embedding into platforms requiring fast, contextual, and human-like dialogue.

Pricing Model

DeepSeek uses a token-based pricing model:

- **Input Tokens (Cache Hit):** $0.014 per 1M tokens (Standard: $0.07)
- **Input Tokens (Cache Miss):** $0.14 per 1M tokens (Standard: $0.27)
- **Output Tokens:** $0.28 per 1M tokens (Standard: $1.10)

This structure supports affordable scaling and efficient use in both development and production environments.

Fig. 6.4. DeepSeek Logo

Architecture Implementation The following diagram illustrates the complete architecture for integrating DeepSeek AI into our chatbot system, showcasing the interaction between the Next.js frontend, Laravel backend, and DeepSeek API:

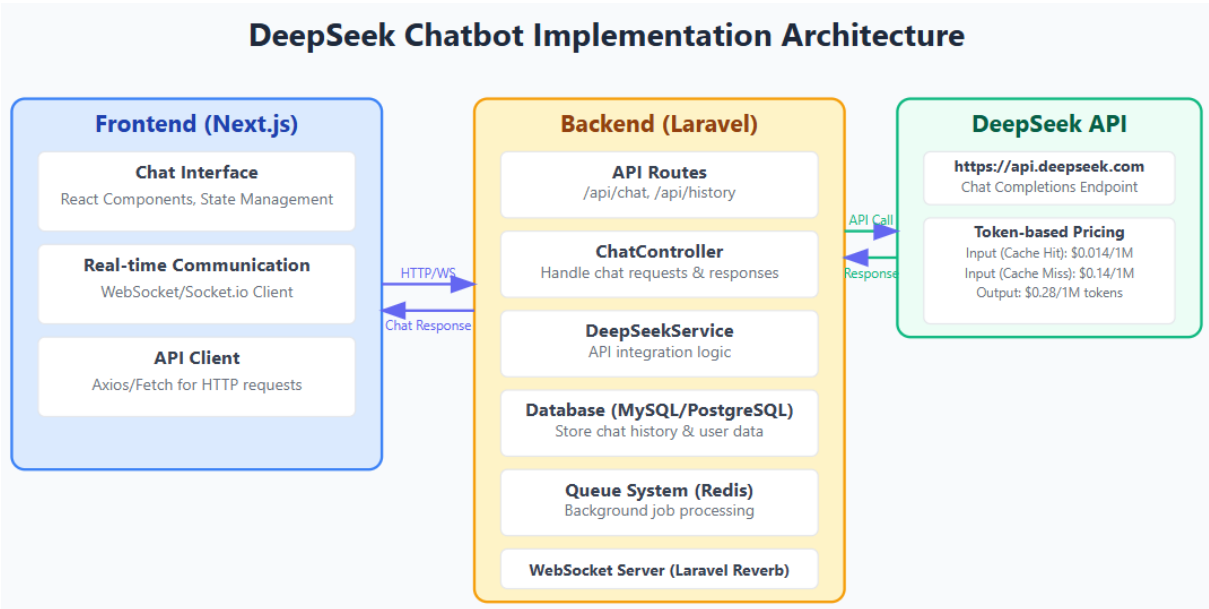
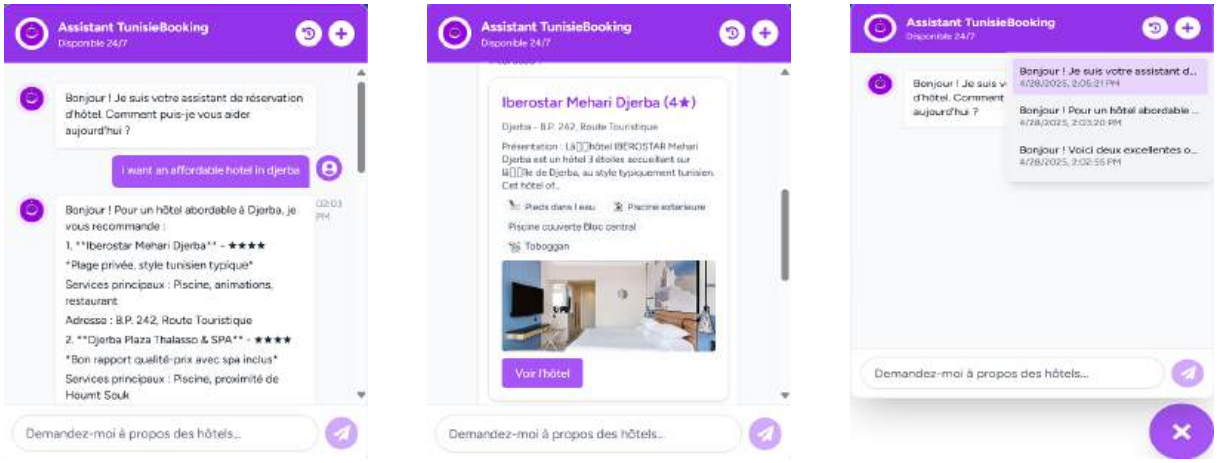


Fig. 6.5. DeepSeek Chatbot Implementation Architecture

6.5 Implementation

ChatBot Interfaces:

The ChatBot interface serves as an intuitive assistant for hotel booking. It allows users to quickly receive personalized hotel suggestions based on their preferences, displaying key information such as services, location, star rating, and amenities, with the ability to access more detailed views. In addition, the chatbot maintains a complete conversation history, capturing all user interactions with timestamps for easy reference. This helps users revisit previous recommendations and conversations at any time. Together, these interfaces provide a smooth and efficient experience for exploring hotel options, making reservations, and managing previous interactions.



(a) ChatBot interface (1) (b) ChatBot interface (2) (c) ChatBot history

Fig. 6.6. ChatBot interfaces

Client Dashboard Interface:

The client dashboard serves as the main entry point to the client space, offering a clean and intuitive navigation system. At the top, users are greeted with a personalized welcome message and a set of clearly labeled navigation buttons: Dashboard, Searches, Reservations, Favorites, and Profile. Each button leads to a dedicated section where users can access recent searches, upcoming or past reservations, preferred hotels, and personal profile settings. This unified interface provides clients with streamlined access to all key functionalities, enhancing the user experience and allowing efficient management of their bookings and preferences.

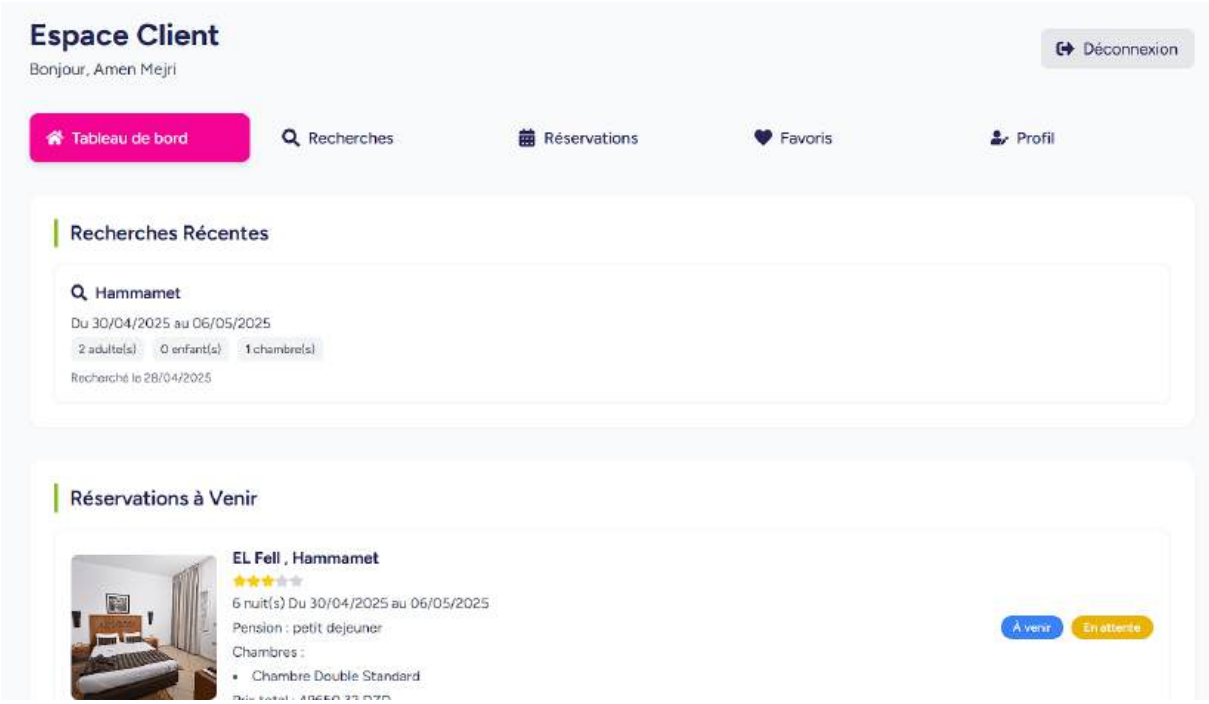


Fig. 6.7. Client Space Home

Search History Interface:

The Search history interface provides a clear view of the search history for hotel bookings. It lists past searches, including details on guests, rooms, and search date.

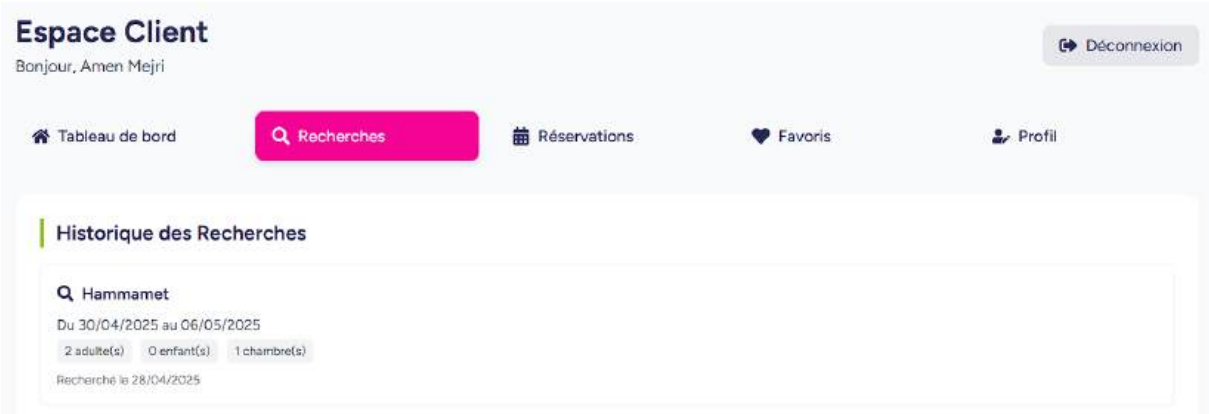


Fig. 6.8. Client Space Search History

Reservations History Interface:

The Reservations interface displays a user's booking history. Details past reservations, including dates, room types, and total cost, with a status indicator.

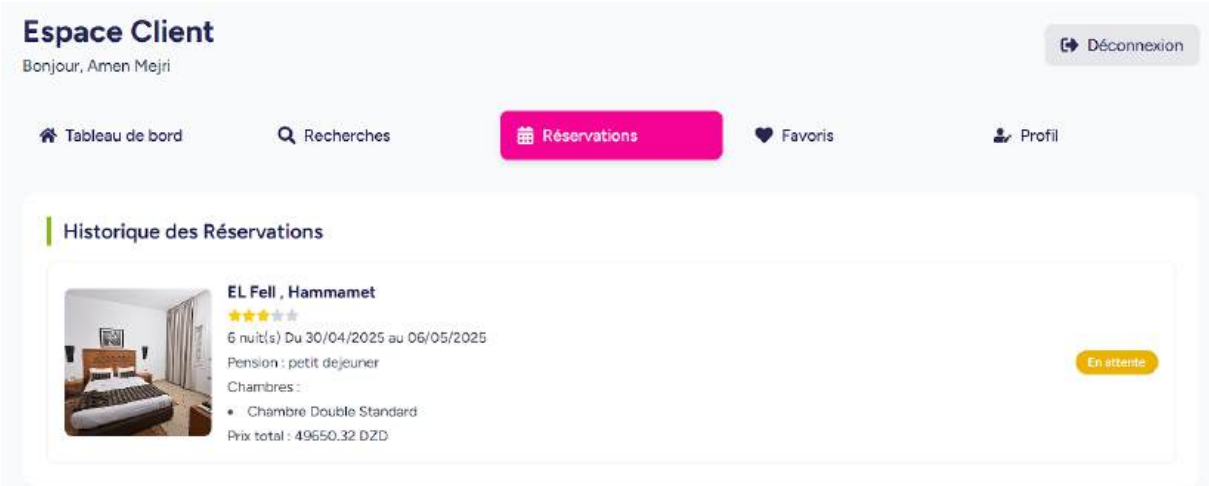


Fig. 6.9. Client Space Reservations

Favorite Hotels Interface:

The Favorite Hotels interface displays the user's favorite hotels and saved options showing star ratings and location, with an easy way to revisit preferred accommodations.

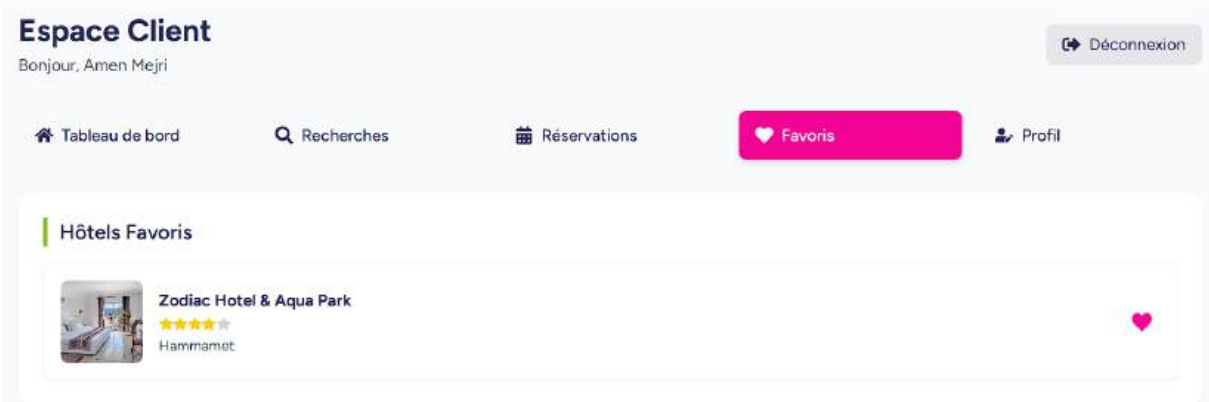


Fig. 6.10. Client Favorite Hotels

Edit Profile Interface:

The Profile interface allows users to manage their personal information seamlessly. It provides a form to update profile details such as name, email, phone number, and country, with options to save or cancel changes.

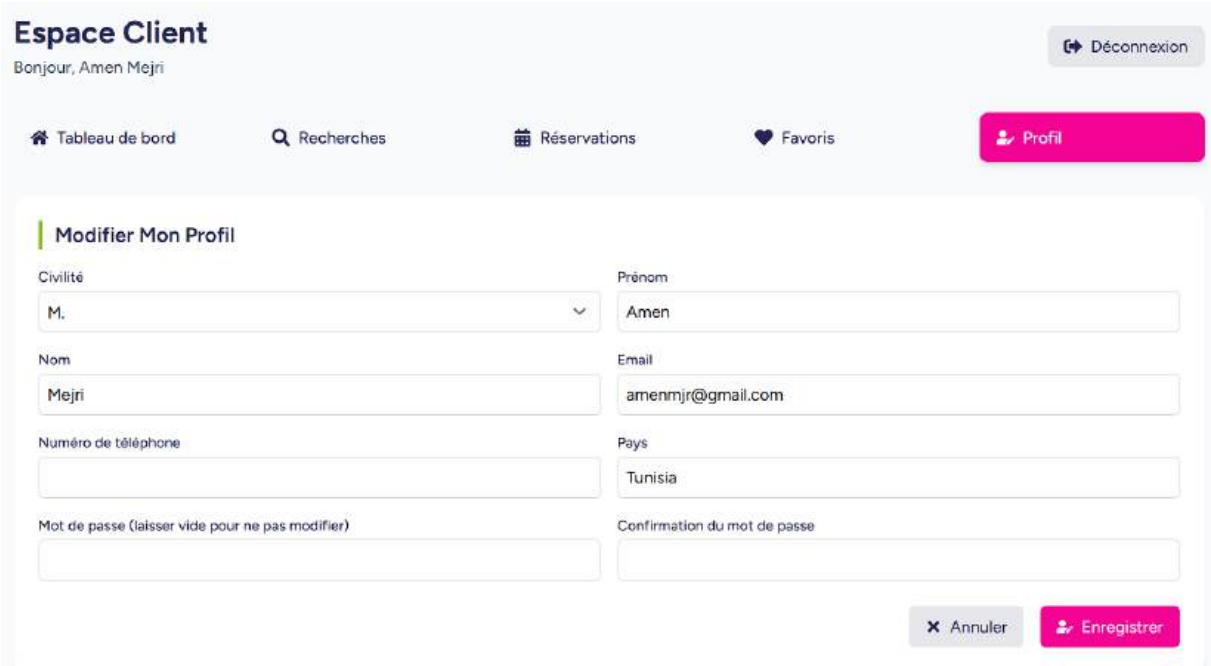


Fig. 6.11. Client Profile

Conclusion

Sprint 4 focused on enhancing the user experience for clients by implementing a comprehensive personal dashboard, favorites management, and integrating an AI-powered chatbot for personalized booking assistance. These features significantly improve platform usability and engagement by offering convenient access to key functionalities and intelligent support. The successful completion of Sprint 4 rounds out the core capabilities of the system, paving the way for future enhancements and refinements.

Chapter 7

Deployment Phase

Contents

Introduction **74**

7.1 Significance of Deployment **74**

7.2 Docker Containerization **75**

7.2.1 Docker Compose & Dockerfile 76

7.3 Hosting Provider Comparison **78**

7.4 Automated Deployment via Coolify **79**

7.5 Deployment Architecture and Advantages **82**

Conclusion **83**

Introduction

The deployment phase represents a critical milestone in the lifecycle of the TunisieBooking platform. This chapter outlines how the application transitions from a development environment to a stable, production ready infrastructure. We emphasize the importance of deployment, the use of Docker for containerization, the rationale behind selecting OVH Cloud for hosting, and the implementation of automated deployment via Coolify. These tools and practices collectively ensure a robust, scalable, and secure deployment strategy.

7.1 Significance of Deployment

Deployment involves publishing a software system to a live environment, making it accessible to real users. For TunisieBooking, an effective deployment process is key to:

- **High Availability:** Ensure continuous access with minimal downtime.
- **Scalability:** Adapt the infrastructure to accommodate increased user demand.
- **Streamlined Updates:** Deliver new features and patches efficiently.
- **Enhanced Security:** Protect user data through secure environments.

A well-executed deployment bridges development and user experience, ensuring reliability and performance.

7.2 Docker Containerization

Overview

Docker is a containerization platform that packages applications along with their dependencies. This ensures consistent behavior across development, testing, and production stages.

Why Docker for TunisieBooking?

- **Consistency:** Identical environments across all stages.
- **Isolation:** Separate containers for each service (frontend, backend, database).
- **Scalability:** Easily scale services independently.
- **Portability:** Run on any Docker-compatible system.

Container Composition

The architecture consists of three key containers:

- **Frontend:** Next.js application responsible for the UI.
- **Backend:** Laravel API handling business logic.
- **Database:** MySQL container managing persistent storage.

Fig. 7.1. Docker Logo

7.2.1 Docker Compose & Dockerfile

Backend – `docker_compose.yml`

Listing 7.2 shows the updated `docker_compose.yml` file used to run the Laravel backend and MySQL database. The Laravel application is served directly via the Artisan server on port 8000 without requiring an Nginx reverse-proxy. The services use named volumes for persistent MySQL storage, and all required environment variables are securely passed through `.env` substitution. This lightweight setup is optimal for development and simple production environments.

```yaml
version: '3.8'

services:
  app:
    build: .
    ports:
      - "8000:8000"
    volumes:
      - .:/var/www
    environment:
      - APP_KEY=${APP_KEY}
      - DB_HOST=db
      - DB_DATABASE=${DB_DATABASE}
      - DB_USERNAME=${DB_USERNAME}
      - DB_PASSWORD=${DB_PASSWORD}
    depends_on:
      - db
    command: php artisan serve --host=0.0.0.0 --port=8000

  db:
    image: mysql:8.0
    environment:
      - MYSQL_DATABASE=${DB_DATABASE}
      - MYSQL_USER=${DB_USERNAME}
      - MYSQL_PASSWORD=${DB_PASSWORD}
      - MYSQL_ROOT_PASSWORD=${DB_ROOT_PASSWORD}
    volumes:
      - db_data:/var/lib/mysql

volumes:
  db_data:
```

Fig. 7.2. `docker_compose.yml` used for Laravel and MySQL containers

Backend – `Dockerfile`

Figure 7.3 displays the Dockerfile used to build the Laravel backend image. It starts from the `php:8.2-fpm` base image, installs necessary PHP extensions, and copies the Composer binary. The Laravel application code is placed in the container, dependencies are installed with production optimization, and file permissions are properly set. Port 8000 is exposed to allow external access via Artisan's development server.

```
FROM php:8.2-fpm

RUN apt-get update && apt-get install -y \
    git curl libpng-dev libonig-dev libxml2-dev zip unzip && \
    docker-php-ext-install pdo_mysql mbstring exif pcntl bcmath gd

COPY --from=composer:latest /usr/bin/composer /usr/bin/composer

WORKDIR /var/www
COPY . /var/www
RUN composer install --optimize-autoloader --no-dev
RUN chown -R www-data:www-data /var/www && chmod -R 755 /var/www/storage

EXPOSE 8000
```

Fig. 7.3. Dockerfile for the Laravel backend container

Frontend – `docker_compose.yml`

Listing 7.4 shows the `docker_compose.yml` configuration for the Next.js frontend. The service listens on port 3000, mounts the application code into the container for easy updates, and exposes the public API URL through an environment variable. The container starts in production mode with the command `npm run start`.

```
version: '3.8'

services:
  nextjs:
    build: .
    ports:
      - "3000:3000"
    environment:
      - NODE_ENV=production
      - NEXT_PUBLIC_API_URL=${NEXT_PUBLIC_API_URL}
    volumes:
      - .:/app
    command: npm run start
```

Fig. 7.4. `docker_compose.yml` used for the Next.js frontend

Frontend – `Dockerfile`

Figure 7.5 presents the Dockerfile that builds the production ready Next.js image. It starts from the lightweight `node:18-alpine` base, installs only production dependencies, compiles the application with `npm run build`, exposes port 3000, and launches the server with `npm run start`. This approach produces a compact, secure container well-suited for cloud deployment.

```
FROM node:18-alpine

WORKDIR /app

COPY package*.json ./
RUN npm install --production

COPY . .
RUN npm run build

EXPOSE 3000
CMD ["npm", "run", "start"]
```

Fig. 7.5. Dockerfile for the Next.js frontend container

7.3 Hosting Provider Comparison

TunisieBooking is deployed on a Virtual Private Server (VPS) provided by OVH Cloud. The following table compares OVH Cloud with two popular alternatives: DigitalOcean and AWS EC2.

Criteria	**OVH Cloud (Chosen)**	**DigitalOcean**	**AWS EC2**
Plan	Comfort VPS	Basic Droplet	t3.medium instance
CPU / RAM	4 vCores / 4–16 GB RAM (upgradable)	1 vCore / 2 GB RAM	2 vCores / 4 GB RAM
Storage	80–320 GB NVMe SSD	50 GB SSD	EBS (variable)
Bandwidth	1 Gbps unmetered	2 TB per month	Variable, metered
Monthly Cost	$15.98	$12.00	Approx. $30.00
Advantages	Affordable, fast storage, unmetered bandwidth	Simple pricing, easy to use	High scalability, global reach
Drawbacks	Fewer global data centers	Lower resources for the price	Expensive, complex setup

Tab. 7.1. Comparison of Hosting Providers for TunisieBooking

OVH Cloud was selected for its cost efficiency, reliable performance, and simplicity making it an ideal choice for a project of this scale. Although AWS offers greater scalability, its pricing and complexity were unnecessary for current needs. DigitalOcean, while user friendly, offered fewer resources at a similar price point.

7.4 Automated Deployment via Coolify

To implement Continuous Integration and Continuous Deployment (CI/CD), the project uses **Coolify**, a modern, self-hosted DevOps platform.

CI/CD Workflow

Each push to the main branch initiates a series of automated tasks:

1. Pull the latest code from GitHub.

2. Build updated Docker images for each service.
3. Run automated tests.
4. Deploy all updated containers to the production environment.

Fig. 7.6. Coolify Logo

Coolify Dashboard and Configuration

The image shows the main dashboard of Coolify, a self-hosted platform used to deploy and manage web applications, databases, and other services. The interface presents a clean and organized layout with several resource cards displayed in a grid format. Each card represents a service such as a web application, database, or background service, showing its current status (like running or stopped), the type of service, and quick action buttons for managing it. At the top, there is a prominent button labeled ”Create new resource”, allowing users to easily add new deployments. This dashboard gives users full control over their infrastructure, making it simple to monitor and manage different environments from a single interface.

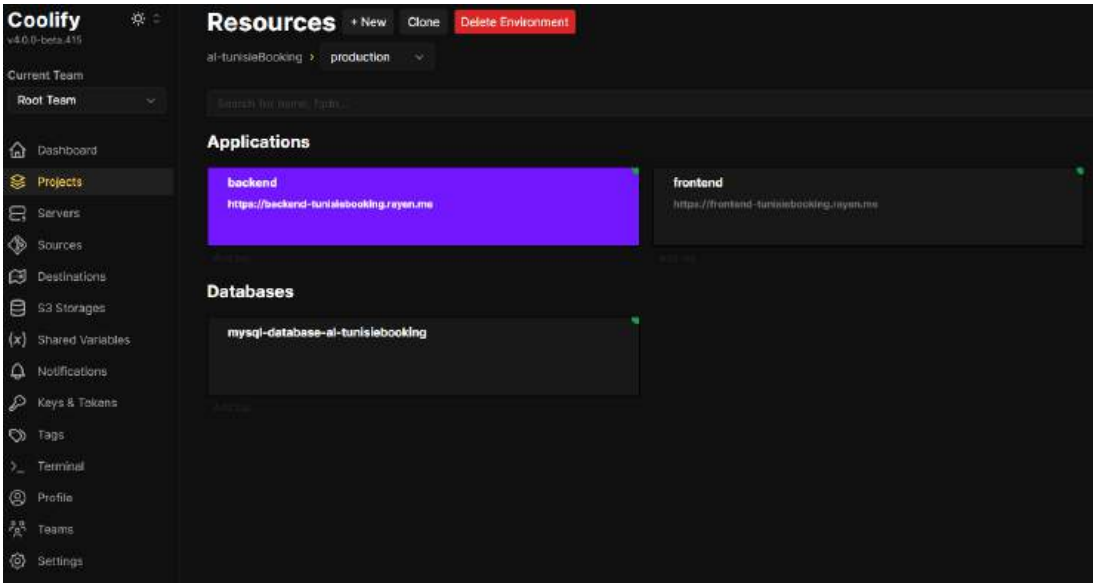


Fig. 7.7. Coolify dashboard overview showing all running containers

When a specific resource is opened in Coolify, such as the *frontend* or *backend*, the user accesses a comprehensive configuration dashboard that allows full control over the service. Key features include:

- **General Settings:** Modify resource name, build pack, and domain.
- **Service Control:** Redeploy, restart, or stop containers.
- **Advanced Features:**
  - Secure management of environment variables.
  - Persistent storage with volume attachment.
  - Git integration for continuous deployment.
  - Scheduled tasks and webhooks.
  - Real-time metrics and health checks.
  - Rollback to previous versions.
  - Web terminal access for debugging.

Coolify centralizes deployment management and monitoring, making it an ideal PaaS (Platform as a Service) for modern DevOps practices.

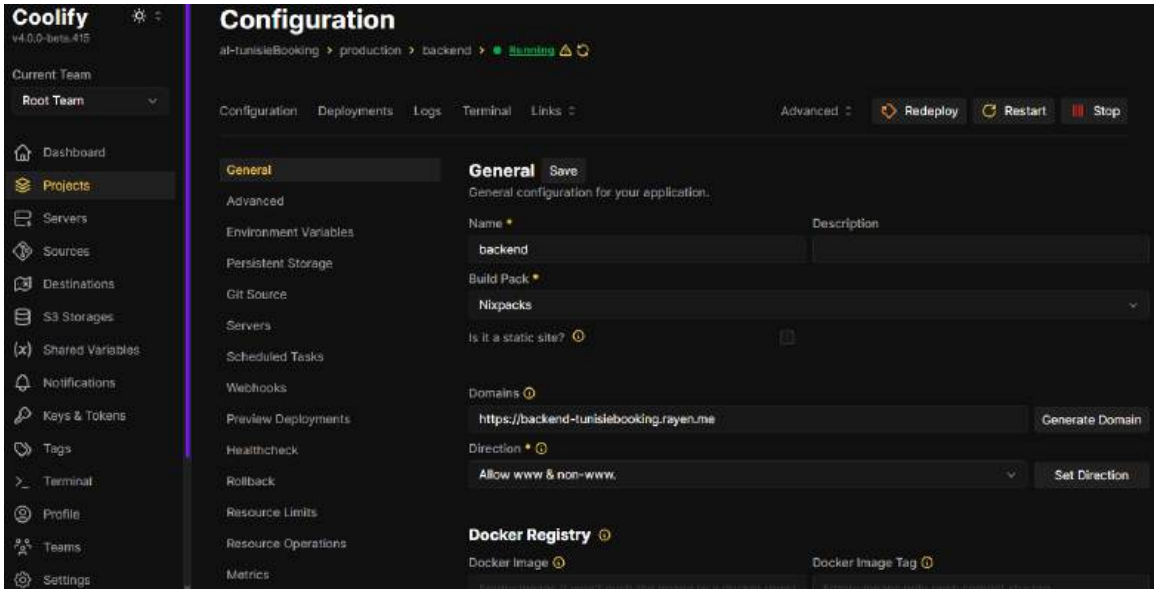


Fig. 7.8. Backend container configuration

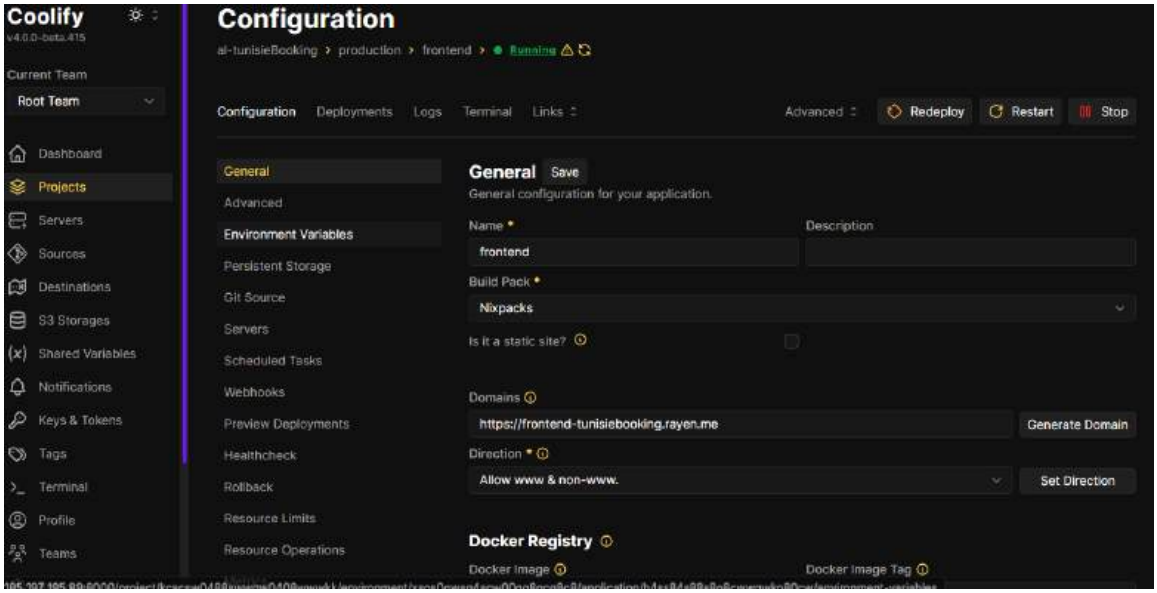


Fig. 7.9. Frontend container configuration

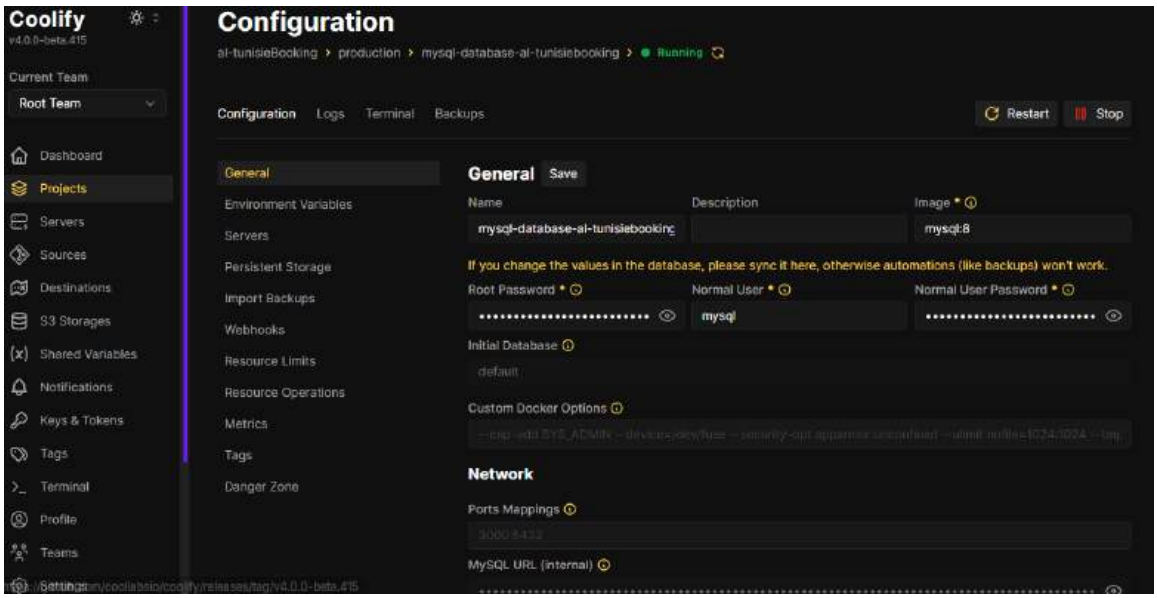


Fig. 7.10. Database container configuration

7.5 Deployment Architecture and Advantages

Fig. 7.11. Preview of the Deployed Website on testal.tunisiebooking.com

System Architecture

The production environment consists of:

- **Frontend:** https://testal.tunisiebooking.com
- **Backend:** https://apial.tunisiebooking.com
- **Database:** MySQL container (internal and secure)

Key Benefits

- **Automation:** CI/CD pipeline accelerates delivery.
- **Isolation:** Modular services simplify maintenance.
- **Security:** Secrets and configs are safely managed.
- **Maintainability:** Rollbacks and updates are effortless.

Conclusion

The deployment phase successfully leveraged Docker, OVH Cloud, and Coolify to build a scalable, secure, and automated production environment. This infrastructure not only enhances performance and reliability but also lays a strong foundation for future upgrades and growth of the TunisieBooking platform.

General Conclusion & Perspectives

This project resulted in a modern, scalable hotel booking platform for Algerian travelers, developed in collaboration with **TunisieBooking**. Guided by the Scrum methodology, the work was structured into four sprints.

The first sprint focused on authentication and user management. The second introduced hotel search, detail viewing, and availability handling. The third covered booking functionality and role-based personalization. The fourth sprint enhanced the client space and integrated an AI chatbot using DeepSeek.

Built with Laravel (backend), Next.js and Tailwind CSS (frontend), and MySQL, the platform ensures performance and maintainability. Deployment with Docker and Coolify on OVH Cloud enabled scalability and efficient infrastructure management.

Perspectives

Potential future enhancements include:

- Adding services like flights, rentals, and tours.
- Developing a native mobile app.
- Improving the chatbot with personalization and multilingual support.
- Supporting multiple currencies and languages for international expansion.
- Boosting performance with caching and load balancing.

Final Remarks

The project met its goals and established a solid foundation. With its modern tech stack and user centered design, TunisieBooking is well-equipped for future growth and innovation.

Webography

[1] TunisieBooking logo:
https://tn.tunisiebooking.com/
Accessed: February 8, 2025.

[2] Waterfall Methodology:
https://technologyadvice.com/blog/project-management/what-is-waterfall-project-management/
Accessed: February 10, 2025.

[3] Scrum Methodology:
https://medium.com/javanlabs/mengenal-scrum-dalam-project-manajemen-a5f660d68644
Accessed: February 10, 2025.

[4] Kanban Methodology:
https://ar.inspiredpencil.com/pictures-2023/kanban-process
Accessed: February 10, 2025.

[5] 2TUP Methodology:
https://www.researchgate.net/
Accessed: February 10, 2025.

[6] MySQL logo:
https://pngimg.com/uploads/mysql/mysql_PNG29.png
Accessed: February 12, 2025.

[7] Laravel PHP logo:
https://logospng.org/download/laravel/logo-laravel-1024.png
Accessed: February 12, 2025.

[8] Next.js logo:
https://th.bing.com/th/id/OIP.xe-ZAgtlBmBzxubtZ8_HVgHaHa?rs=1&pid=ImgDetMain
Accessed: February 12, 2025.

[9] Tailwind CSS logo:
https://upload.wikimedia.org/wikipedia/commons/thumb/d/d5/Tailwind_CSS_Logo.svg/1024px-Tailwind_CSS_Logo.svg.png?20230715030042
Accessed: February 12, 2025.

[10] MVC Architecture:
https://www.interviewbit.com/blog/mvc-architecture/
Accessed: February 15, 2025.

[11] DeepSeek logo:
https://www.deepseek.com/
Accessed: April 20, 2025.

[12] Docker logo:
https://www.Docker.com/
Accessed: April 21, 2025.

[13] Coolify logo:
https://www.Coolify.io/
Accessed: April 21, 2025.

Abstract

This project involved the design and development of a hotel booking web platform tailored for an Algerian travel agency, in collaboration with TunisieBooking, a Tunisian leader in online reservations. The platform offers a seamless and secure experience for Algerian tourists booking hotels in Tunisia, aggregating data from multiple providers and offering real-time availability through API integrations.

Built using a modern tech stack including Laravel, Next.js, MySQL, and Docker the solution ensures scalability, reliability, and performance. The system was developed using an agile Scrum methodology, which allowed for iterative delivery and continuous improvements throughout the development sprints.

By providing personalized recommendations, intuitive navigation, and a responsive design, the platform enhances the travel planning experience and serves as a strategic tool for both the agency and its partners.

Keywords: Hotel booking, Laravel, Next.js, MySQL, Docker, Agile methodology.

Résumé

Ce projet porte sur la conception et le développement d'une plateforme web de réservation d'hôtels, réalisée pour une agence de voyages algérienne en partenariat avec TunisieBooking, leader tunisien dans le domaine des réservations en ligne. La plateforme permet aux touristes algériens de réserver facilement des hôtels en Tunisie, en offrant une agrégation de données provenant de plusieurs fournisseurs et une disponibilité en temps réel via des intégrations API.

Développée avec des technologies modernes telles que Laravel, Next.js, MySQL et Docker, la solution garantit évolutivité, fiabilité et performance. Le projet a été mené selon une méthodologie agile Scrum, permettant un développement itératif et une amélioration continue à travers les différents sprints.

En proposant des recommandations personnalisées, une navigation intuitive et un design responsive, cette plateforme améliore l'expérience de planification de voyage et constitue un véritable outil stratégique pour l'agence et ses partenaires.

Mots clés : Réservation d'hôtels, Laravel, Next.js, MySQL, Docker, Méthodologie agile.